Doc. no. P2150R0
Date: 2020-04-14
Project: Programming Language C++
Audience: Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Down with ~~typename~~ in the Library!

Table of Contents

Revision History

Revision 0

Original version of the paper for the 2020 April mailing.

1 Abstract

The paper P0634R3, adopted at Jacksonville 2018, simplified the rules requiring the use of the `typename` keyword where no other interpretation might be permitted. This paper suggests editorially applying the practice consistently, throughout the library clauses, so that

unnecessary `typenames` are omitted.

2 Stating the problem

As the C++ language evolves, simpler and cleaner ways of expressing ideas become available. Over time, the Library specification style becomes outdated and cumbersome, unless we try to stay on top of relevant changes when they are adopted. One example is P0634R3 adopted at Jacksonville (2018) that makes many uses of the `typename` keyword redundant, when used to disambiguate types from expressions in dependent contexts by the library specification. While we could retain this usage for clarity, informed readers of the specification will start wondering why the redundant keyword is present, and start looking for the ambiguity that the keyword resolves, where none exists.

It should be noted that a related paper was rejected by the Evolution Working Group that could have had an impact on a subset of these now redundant `typenames`. Specifically, P0945R0 would have reintroduced a context where `typename` is a necessary disambiguator in alias template declarations. As this proposal was rejected for C++20, it could not be resurrected without making breaking changes to valid user programs, and should no longer be a concern for recommending best practice for the library specification.

2.1 Why Is `typename` Necessary?

As a quick refresher before jumping into a case-by-case analysis, let's remind ourselves why the `typename` keyword is needed in the first place.

The need for the `typename` keyword falls out of the two-phase name lookup scheme for templates, adopted for C++98. In the first phase of name lookup, if a template parses a dependent name, it cannot know if that name refers to a type, a template, or a value. In order to catch more errors while parsing an uninstantiated template (phase 1) we assume that any dependent name denotes a value, and so can be consumed by an expression. If a type or a template is required, that is a diagnosable error, indicating a likely problem in the template. If a template is required, we can disambiguate with the `.template` or `::template` syntax. Likewise, if a type is required, we can disambiguate with the `typename` keyword.

Prior to P0634R3 the grammar would always assume that a dependent name denoted a value, without regard to context. However, that paper simplifies usage in the cases where the grammar can unambiguously expect a type, making use of the `typename` keyword to disambiguate types from values optional in such contexts.

3 Propose Solution

The recommendation of this paper is to adopt editorial guidelines for the library clauses of the standard (16-32 plus Annex D) to eliminate redundant use of the `typename` keyword. Once applied, this should become the expected form of wording for subsequent LWG reviews, although there may be a meeting or two lag where such changes to incoming papers are applied editorially, to avoid asking LWG to spend additional time re-reviewing largely approved wording.

Below, we analyze a set of common code patterns where `typename` is required by the C++17 grammar that may (or may not) be affected by the application of P0634R3. This paper attempts to evaluate a wide variety of syntax patterns to cover cases not yet present in library wording, in case guidance for future papers is needed. It is intended that this paper could serve as a style guide for future papers regarding appropriate use of `typename`, and may be updated as additional questions of usage arise - unless a more broad reaching paper of library coding conventions is adopted.

At the time of writing, two compilers implement the needed language feature: the EDG front end and the (not yet released) gcc 10 compiler. All code has been tested against a recent build of the pending gcc 10 compiler. If there is any bad analysis in the examples below, we should likely file bug reports against this compiler (and amend the paper!)

Note that it would be quite reasonable to adopt just a selected subset of the options to remove redundant `typenames` below. For simplicity, this first draft of the paper assumes that all possible options will be taken, and defers finding a markup for which valid options are not taken up until it is needed.

All wording is relative to the initial working draft for C++23, N4861.

3.1 Patterns that Preserve `typename`

First we will examine the places where the language is not yet ready to drop the requirement to use `typename`, with a relevant example in each case. Ideally examples will quote from the standard library, otherwise fresh examples are created where the syntax pattern is not yet used within the library clauses of the standard, to preserve the knowledge in case it becomes relevant to wording future proposals.

3.1.1 Preserve `typename` in *Non-member* Function Parameter Declarations

Relevant examples from the standard library:

31.2 Header `<atomic>` synopsis [atomics.syn]

```
namespace std {
  // 31.9, non-member functions
  template<class T>
    bool atomic_is_lock_free(const volatile atomic<T>*) noexcept;
  template<class T>
    bool atomic_is_lock_free(const atomic<T>*) noexcept;

  template<class T>
    void atomic_store(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    void atomic_store(atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    void atomic_store_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
  template<class T>
    void atomic_store_explicit(atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
}
```

Rationale: I was given the following by Richard Smith which is far better quoted directly than paraphrasing in in my own words:

> This definitely isn't a case the "down with typename" paper missed accidentally: we need these `typenames` (at least when they're part of the **first** parameter) in order to distinguish between a function template declaration and a variable template declaration:
>
> ```
> template<typename T> T x(typename T::foo); // function template
> template<typename T> T x(T::foo);          // variable template
> ```
>
> and we very consciously did not want to make a special rule for "subsequent parameters".

Note that there are no variable template class members, so this ambiguity does not arise for member functions.

Also note that this restriction does not apply to the return type of a function, nor does it apply to a function definition with a qualified name, so:

```
template <class T>
struct Identity {
   using type = T;
};

namespace ns {
  template <class T>
  typename Identity<T>::type clone(typename Identity<T>::type dummy);
}

template <class T>
typename Identity<T>::type ns::clone(typename Identity<T>::type dummy) {
   return typename Identity<T>::type{};
}
```

3.1.2 Preserve `typename` Instantiating Template Arguments

Relevant examples from the standard library:

20.11.1.5 Specialized algorithms [unique.ptr.special]

```
template <class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

4. Let `CT` denote

```
common_type_t<typename unique_ptr<T1, D1>::pointer,
              typename unique_ptr<T2, D2>::pointer>
```

5. *Mandates:*
    1. — `unique_ptr<T1, D1>::pointer` is implicitly convertible to `CT` and

2. — `unique_ptr<T2, D2>::pointer` is implicitly convertible to `CT`.

29.9.2 Class template `basic_filebuf` [filebuf]

5. In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as `a_codecvt` in following subclauses, obtained as if by

```
const codecvt<charT, char, traits::state_type>& a_codecvt =
  use_facet<codecvt<charT, char, typename traits::state_type>>(getloc());
```

Rationale: When instantiating a template, the compiler does not look at the template declaration to guide whether the supplied arguments should be parsed as a type, as a template, or as an expression (value). Hence, the `typename` keyword is not optional for any template instantiation.

Suggestion by Richard Smith to further revise Core language to support template arguments

When parsing a template-argument for which the template-name resolves to a specific non-function template that is not a member of an unknown specialization, parse the *template-argument* as a type if the corresponding parameter is a type parameter, as an expression if the corresponding parameter is a non-type template parameter, and as a template name otherwise. (We'll still need the "parse it as whatever you can, and try to deal with the mess later" approach for the dependent template cases, or when the *template-name* names a function template.)

This is not entirely backwards-compatible (there are obscure cases where the existence of a type template causes us to parse a `<` as being part of a *template-id*, but we later resolve that *template-id* as a function template name), and it creates a jarring dissonance between function templates and other kinds of templates (and we still don't get to make `dynamic_pointer_cast<...>` accept the same template-argument syntax as `dynamic_cast<...>`). These concerns (particularly the latter) make me think that this isn't an obviously good change, but on balance it seems worthwhile.

3.1.3 Preserve `typename` for Partial Template Specializations

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template<class Type, class Iterator>
struct Host {};

template<class Type>
struct Host<Type, typename Type::iterator> {};
```

Rationale: Just as when instantiating a template, the compiler does not look at the template declaration to guide whether the supplied arguments should be parsed as a type, as a template, or as an expression (value). Hence, the `typename` keyword is not optional for template partial specializations either.

3.1.4 Preserve `typename` for Arguments to Concepts

Relevant example from the standard library:

27.5.6 Comparisons [time.duration.comparisons]

```
template<class Rep1, class Period1, class Rep2, class Period2>
    requires three_way_comparable<typename CT::rep>
  constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
                             const duration<Rep2, Period2>& rhs);
```

7. *Returns:* `CT(lhs).count() <=> CT(rhs).count()`.

Rationale: Just as when instantiating a template, the compiler does not look at the concept declaration to guide whether the supplied arguments should be parsed as a type, as a template, or as an expression (value). Hence, the `typename` keyword is not optional when using a concept.

3.1.5 Preserve `typename` in Deduction Guides

Relevant examples from the standard library:

> **21.3.2 Class template `basic_string` [basic.string]**
>
> ```
> namespace std {
>   template<class charT, class traits = char_traits<charT>,
>            class Allocator = allocator<charT>>
>   class basic_string {
>     // ...
>   };
>
>
>   template<class InputIterator,
>            class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
>     basic_string(InputIterator, InputIterator, Allocator = Allocator())
>       -> basic_string<typename iterator_traits<InputIterator>::value_type,
>                       char_traits<typename iterator_traits<InputIterator>::value_type>,
>                       Allocator>;
>
>   template<class charT, class traits,
>            class Allocator = allocator<charT>>
>     basic_string(basic_string_view<charT, traits>,
>                  typename see below::size_type, typename see below::size_type,
>                  const Allocator& = Allocator())
>       -> basic_string<charT, traits, Allocator>;
> }
> ```

Rationale: The `typename` keyword remains non-optional at all points in the deduction guide grammar. In the template-head, it is as essential as in any other template declaration. In the guiding declaration, it remains essential for all the constructor parameters, as this appears in the language grammar similar to a non-member function. Finally, it remains essential in the guided deduction, as that acts like any other template instantiation.

3.1.6 Preserve `typename` in Variable Template definitions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
struct Indirect {
   using type = T;
};

template <class T>
auto default_value = typename Indirect<T>::type{};
```

Rationale: Variables are initialized by expressions, which is what a dependent name is expected to produce absent the `typename` keyword. Hence, its use remains non-optional in this case. However, see 3.2.3 for the declaration on the left-hand hand of the initializer.

3.1.7 Preserve `typename` for Local Variable Declarations

Relevant example from the standard library:

> **28.3.1 `Class locale` [locale]**
>
> 3. [*Example:* An iostream `operator<<` might be implemented as:[257]
>
> ```
> template<class charT, class traits>
> basic_ostream<charT, traits>&
> operator<< (basic_ostream<charT, traits>& s, Date d) {
>   typename basic_ostream<charT, traits>::sentry cerberos(s);
>   if (cerberos) {
>     tm tmbuf; d.extract(tmbuf);
>     bool failed =
>       use_facet<time_put<charT, ostreambuf_iterator<charT, traits>>>(
>         s.getloc()).put(s, s, s.fill(), &tmbuf, 'x').failed();
>     if (failed)
>       s.setstate(s.badbit);     // might throw
>   }
> ```

```
    return s;
  }
```

— end example]

Rationale: Variables are initialized by expressions, which is what a dependent name is expected to produce absent the `typename` keyword. Hence, its use remains non-optional in this case.

3.1.8 Preserve `typename` when Constructing Temporary Objects for Return Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
struct Indirect {
   using type = T;
};

template <class T>
auto factory() -> Indirect<T>::type {
   return typename Indirect<T>::type{};
}
```

Rationale: Return values are necessarily expressions, and so continue to require the `typename` keyword, as this is the very reason it exists.

3.1.9 Preserve `typename` when Constructing Temporary Objects for Function Arguments

Relevant example from the standard library:

25.10.3 Reduce [reduce]

```
template<class InputIterator>
  typename iterator_traits<InputIterator>::value_type
    reduce(InputIterator first, InputIterator last);
```

1. *Effects:* Equivalent to:

```
return reduce(first, last,
              typename iterator_traits<InputIterator>::value_type{});
```

Rationale: Passing a temporary value to a function call necessarily creates an expression, and so continues to require the `typename` keyword, as this is the very reason it exists.

3.1.10 Preserve `typename` for Default Function Arguments

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
struct Indirect {
   using type = T;
};

template <class T>
void fun(typename Indirect<T>::type = typename Indirect<T>::type{}) {}
```

Rationale: Default function arguments are just another way of packaging up the value to pass, and so have the same concerns as any other expression to pass a function argument, as above. Hence, the `typename` keyword remains required.

3.1.11 Preserve `typename` for Default Member Initializers

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template<class T>
struct host {
```

```
   T::type d_value = typename T::type{};
};
```

Rationale: Default member initializers are necessarily expressions, and so continue to require the `typename` keyword to construct temporary arguments.

3.1.12 Preserve `typename` for `alignof` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void func() {
   auto x = alignof(typename T::type);
}
```

Rationale: An `alignof` expression requires that its argument be a *type-id*, and so in principle the requirement for `typename` could be relaxed in this context by a future standard. However, this is (indirectly) the subject of core issue #1008, an extension request that was discussed recently by EWG and there may well be a forthcoming proposal for `alignof` to accept expressions, much like `sizeof`, in which case the `typename` keyword would become necessary to disambiguate the two cases.

3.1.13 Preserve `typename` within `decltype` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void func() {
   using x = decltype(typename T::type{});
}
```

Rationale: Any syntax that expects an expression continues to require the `typename` keyword, as this is the very reason it exists. Note that this usage of `decltype` is essentially an overly complicated way of spelling the type already clearly identified within the parentheses.

3.1.14 Preserve `typename` for `noexcept` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void func() {
   auto x = noexcept(typename T::type{});
}
```

Rationale: Any syntax that expects an expression continues to require the `typename` keyword, as this is the very reason it exists.

3.1.15 Preserve `typename` for `sizeof` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void func() {
   auto x = sizeof(typename T::type &&);
   auto y = sizeof(typename T::type {});
   auto z = sizeof typename T::type {} ;
}
```

Rationale: Any syntax that expects or admits an expression continues to require the `typename` keyword, as this is the very reason it exists. The form of `sizeof` taking a type name in parentheses must be disambiguated from `sizeof` a plain parenthetical expression.

3.1.16 Preserve `typename` for `throw` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void chuck() {
   throw typename T::type{};
}
```

Rationale: Any syntax that expects an expression continues to require the `typename` keyword, as this is the very reason it exists.

3.1.17 Preserve `typename` for `typeid` Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
#include <typeinfo>

template <class T>
void func() {
   auto x = typeid(typename T::type *);
   auto y = typeid(typename T::type{});
}
```

Rationale: Any syntax that expects or admits an expression continues to require the `typename` keyword, as this is the very reason it exists.

3.1.18 Preserve `typename` for Cast Expressions

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
void func() {
   auto a = static_cast<T>(typename T::type{});
   auto b = const_cast<T const &>(typename T::type{});
   auto c = dynamic_cast<T>(typename T::type{});
   auto d = reinterpret_cast<T>(typename T::type{});
   auto e = (void)typename T::type{};
   auto f = (typename T::type *)nullptr;
}
```

Rationale: The argument to a cast is necessarily an expression, and so continues to require the `typename` keyword, as this is the very reason it exists. However, see 3.2.8 below regarding the angle-bracket portion of this syntax.

3.1.19 Preserve `typename` in Exception Specifications

There is no example in the standard library, so we provide our own example to illustrate the use case below. Note that this construction is particularly obscure, relying on contextual conversion from objects of the dependent type to `bool`.

```
struct Boolish {
   constexpr explicit operator bool() { return true; }
};

template <class T>
struct Indirect {
   using type = Boolish;
};

template <class T>
void func() noexcept(typename Indirect<T>::type{});
```

Rationale: The content of an exception specification is a predicate expression, and so continues to require the `typename` keyword.

3.2 Patterns that Remove `typename`

Next, we will examine the places where the language has sufficient context to make the use of `typename` optional, with a relevant example in each case. Ideally examples will quote from the standard library, otherwise fresh examples are created where the syntax pattern is not yet used within the library clauses of the standard, to preserve the knowledge in case it becomes relevant to wording future proposals.

3.2.1 Remove `typename` from Alias Definitions

Relevant examples from the standard library:

20.15.2 Header `<type_traits>` synopsis [meta.type.synop]

```
namespace std {
template<class T>
  using remove_const_t = typename remove_const<T>::type;
}
```

21.3.2 Class template `basic_string` [basic.string]

```
namespace std {
  template<class charT, class traits = char_traits<charT>,
           class Allocator = allocator<charT>>
  class basic_string {
  public:
    // types
    using traits_type        = traits;
    using value_type         = charT;
    using allocator_type     = Allocator;
    using size_type          = typename allocator_traits<Allocator>::size_type;
    using difference_type    = typename allocator_traits<Allocator>::difference_type;
    using pointer            = typename allocator_traits<Allocator>::pointer;
    using const_pointer      = typename allocator_traits<Allocator>::const_pointer;
    using reference          = value_type&;
    using const_reference    = const value_type&;

    // ...
  };
}
```

22.3.8.1 Overview [deque.overview]

```
namespace std {
  template<class T, class Allocator = allocator<T>>
    class deque {
    public:
      // types
      using value_type             = T;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2
      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using reverse_iterator       = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;

      // ...
    };
}
```

Rationale: When defining a type alias, or alias template, with either `using` or `typedef`, the parser is already primed to expect a type, so `typename` can be safely omitted. This would be by far the most common application of the remove redundant `typenames` from the library principle.

Note that early drafts of this paper were hesitant to recommend striking `typename` in these cases while P0945R0 was still in flight, generalizing the notion of aliases through `using`. That proposal was rejected, and now that C++20 is finalized, it could not return without being a breaking change, so the recommendation to remove `typename` in these cases stands, and is likely the most widely encountered case for a change.

3.2.2 Remove `typename` from Default Template Arguments

Relevant example from the standard library:

27.2 Header `<chrono>` synopsis [time.syn]

```
namespace std {
  namespace chrono {
    // 27.5, class template duration
    template<class Rep, class Period = ratio<1>> class duration;

    // 27.6, class template time_point
    template<class Clock, class Duration = ~~typename~~ Clock::duration> class time_point;
  }
}
```

Rationale: When declaring a template-head with default template parameters, the right-hand side of the `=` is known to require a type (for a type parameter) so there is no ambiguity to discriminate, and the `typename` keyword becomes optional.

3.2.3 Remove `typename` from Variable Template declarations

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template <class T>
struct Indirect {
   using type = T;
};

template <class T>
~~typename~~ Indirect<T>::type default_value = typename Indirect<T>::type{};

template <class T>
~~typename~~ Indirect<T>::type another_value{ typename Indirect<T>::type{} };
```

Rationale: This case is confusingly similar to the non-redundancy highlighted for variable template *definitions* where the `typename` remains essential when providing the value to initialize with. However, the `typename` on the declaration itself is now optional, as illustrated in the example above.

3.2.4 Remove `typename` from Function Return Types

Relevant examples from the standard library:

21.3.1 Header `<string>` synopsis [string.syn]

```
namespace std {

  // ...

  // 21.3.3.5, erasure
  template<class charT, class traits, class Allocator, class U>
    constexpr ~~typename~~ basic_string<charT, traits, Allocator>::size_type
      erase(basic_string<charT, traits, Allocator>& c, const U& value);
  template<class charT, class traits, class Allocator, class Predicate>
    constexpr ~~typename~~ basic_string<charT, traits, Allocator>::size_type
      erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);

  // basic_string typedef names
  // ...
}
```

There is no example in the standard library of a dependent trailing return type using `typename`, so we adapt the previous example for illustration purposes below:

```
template<class charT, class traits, class Allocator, class U>
  constexpr auto erase(basic_string<charT, traits, Allocator>& c, const U& value)
              -> ~~typename~~ basic_string<charT, traits, Allocator>::size_type;
template<class charT, class traits, class Allocator, class Predicate>
  constexpr auto erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);
              -> ~~typename~~ basic_string<charT, traits, Allocator>::size_type;
```

Rationale: The grammar is unambiguous that the return type of a function is indeed a type, so the `typename` keyword is optional in this context.

3.2.5 Remove `typename` from Friend Function Declarations

There is no example in the standard library, so we provide our own example to illustrate the use case below.

```
template<class T>
struct host {
   friend void func(~~typename~~ T::type) {}

   template<class U>
   friend void func(~~typename~~ U::type) {}
};
```

Rationale: The grammar inside a class definition does not suffer from the vexing parse issues of global/namespace scope, even for friend functions, so `typename` can be safely omitted from any friend function parameters.

Note that default function arguments remain constrained to use the `typename` keyword in all scenarios.

3.2.6 Remove `typename` from Member Function Declarations

Relevant example from the standard library:

> **23.5.2.1 Class template `back_insert_iterator` [back.insert.iterator]**
>
> ```
> namespace std {
>   template<class Container>
>   class back_insert_iterator {
>   // ...
>
>   public:
>     using iterator_category = output_iterator_tag;
>     using value_type        = void;
>     using difference_type   = ptrdiff_t;
>     using pointer           = void;
>     using reference         = void;
>     using container_type    = Container;
>
>     constexpr back_insert_iterator() noexcept = default;
>     constexpr explicit back_insert_iterator(Container& x);
>     constexpr back_insert_iterator& operator=(const ~~typename~~ Container::value_type& value);
>     constexpr back_insert_iterator& operator=(~~typename~~ Container::value_type&& value);
>
>     // ...
>   };
> }
> ```

Rationale: The grammar inside a class definition does not suffer from the vexing parse issues of global/namespace scope, so `typename` can be safely omitted from any member function parameters. Similarly, for a member function to be defined outside the class definition, it must be redeclared with a qualified name, which again avoids ambiguous parses, making `typename` optional.

Note that default function arguments remain constrained to use the `typename` keyword in all scenarios.

3.2.7 Remove `typename` from Data Member Declarations

Relevant example from the standard library:

> **22.2.4.1 Overview [container.node.overview]**
>
> ```
> template<unspecified>
> class node-handle {
>   // ...
>
> private:
>   using container_node_type = unspecified;
>   using ator_traits = allocator_traits<allocator_type>;
> ```

```
  typename ator_traits::rebind_traits<container_node_type>::pointer ptr_;
  optional<allocator_type> alloc_;

  // ...
};
```

Rationale: The grammar inside a class definition does not suffer from the vexing parse issues of global/namespace scope, so `typename` can be safely omitted from any data member declarations.

3.2.8 Remove `typename` from C++ Cast Types

Relevant example from the standard library:

> **20.11.3.9 Casts [util.smartptr.shared.cast]**
>
> ```
> template<class T, class U>
>   shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
> template<class T, class U>
>   shared_ptr<T> static_pointer_cast(shared_ptr<U>&& r) noexcept;
> ```
>
> 1. *Mandates:* The expression `static_cast<T*>((U*)nullptr)` shall be well-formed.
> 2. *Returns:*
>
>    ```
>    shared_ptr<T>(R, static_cast<typename shared_ptr<T>::element_type*>(r.get()))
>    ```
>
>   where *R* is `r` for the first overload, and `std::move(r)` for the second.
> 3. [*Note:* The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

Rationale: For each of the four C++ type-casts, the content of the angle brackets specifying what to cast to is unambiguously a type, so the `typename` keyword is optional. This does not apply to C-style casts, as the grammar around parentheses is more nuanced.

3.2.9 Remove `typename` from New Expressions

Relevant example from the standard library:

> **25.11.3 `uninitialized_value_construct` [uninitialized.construct.value]**
>
> ```
> template <class NoThrowForwardIterator>
>   void uninitialized_value_construct(NoThrowForwardIterator first, NoThrowForwardIterator last);
> ```
>
> 1. *Effects:* Equivalent to:
>
>    ```
>    for (; first != last; ++first)
>      ::new (voidify(*first))
>        typename iterator_traits<NoThrowForwardIterator>::value_type();
>    ```

Rationale: It is clear that for a `new` expression to create an object, you must name the type of the object to be created, so there is no ambiguity in the grammar at this point, and the `typename` can be safely omitted.

4 Impact on the Standard

The net effect will be a cleaner standard with less syntactic noise distracting from reading a variety of declarations. In some cases, this will also lead to less line wrapping, further promoting clarity.

There is no risk to the language in adopting the feature, as it is purely editorial in effect. However, there may be a perceived risk in applying such an edit entirely under the purview of editorial oversight, in the case of accidental overreach removing a necessary `typename`.

One subtle consequence is that it might no longer be as easy to copy/paste library declarations out of the standard directly into a standard library implementation, unless the vendor is willing to assume a C++20 baseline for their support. For vendors wishing to maintain ongoing support for customers relying on older dialects of the language, they would have to enter the redundant `typename`s themselves.

5 Tentative Wording (Informative)

The following are a representative set of changes from applying the editorial guidelines above; they are not intended to be complete for review purposes (although a best faith effort hopes they are, outside Annex D!) but to give a fair impression of the kind and scale of changes that are proposed. It is expected that the editors would take this as guidance for bringing the working draft into a good state, rather than perform a line-by-line review by the whole LWG. Note that unlike the sequence of mandating papers for C++20, there are no issues with rewording requiring a more detailed review - the only changes are striking redundant use of a single keyword, not replacing it with anything else.

The editorial conventions for the following wording are:

- `typename` : non-redundant `typename` that must be retained
- ~~`typename`~~ : redundant `typename` that should be excised
- **`typename`** : grammatical use of `typename` other than to disambiguate dependent names

Where a `typename` is retained in a declaration in a header synopsis, the normative specification is not duplicated unless there are additional interactions in that specification wording itself.

17.11.1 Header `<compare>` synopsis [compare.syn]

1. The header `<compare>` specifies types, objects, and functions for use primarily in connection with the three-way comparison operator (7.6.8).

```
namespace std {
  // 17.11.2, comparison category types
  class partial_ordering;
  class weak_ordering;
  class strong_ordering;

  // named comparison functions
  constexpr bool is_eq  (partial_ordering cmp) noexcept { return cmp == 0; }
  constexpr bool is_neq (partial_ordering cmp) noexcept { return cmp != 0; }
  constexpr bool is_lt  (partial_ordering cmp) noexcept { return cmp <  0; }
  constexpr bool is_lteq(partial_ordering cmp) noexcept { return cmp <= 0; }
  constexpr bool is_gt  (partial_ordering cmp) noexcept { return cmp >  0; }
  constexpr bool is_gteq(partial_ordering cmp) noexcept { return cmp >= 0; }

  // 17.11.3, common comparison category type
  template<class... Ts>
    struct common_comparison_category {
      using type = see below;
    };
  template<class... Ts>
    using common_comparison_category_t = typename common_comparison_category<Ts...>::type;

  // 17.11.4, concept three_way_comparable
  template<class T, class Cat = partial_ordering>
    concept three_way_comparable = see below;
  template<class T, class U, class Cat = partial_ordering>
    concept three_way_comparable_with = see below;

  // 17.11.5, result of three-way comparison
  template<class T, class U = T> struct compare_three_way_result;

  template<class T, class U = T>
    using compare_three_way_result_t = typename compare_three_way_result<T, U>::type;

  // 20.14.7.7, class compare_three_way
  struct compare_three_way;

  // 17.11.6, comparison algorithms
  inline namespace unspecified {
    inline constexpr unspecified strong_order  = unspecified;
    inline constexpr unspecified weak_order    = unspecified;
    inline constexpr unspecified partial_order = unspecified;
    inline constexpr unspecified compare_strong_order_fallback  = unspecified;
    inline constexpr unspecified compare_weak_order_fallback    = unspecified;
    inline constexpr unspecified compare_partial_order_fallback = unspecified;
  }
}
```

17.12.2.1 Class template `coroutine_traits` [coroutine.traits.primary]

1. The header `<coroutine>` defines the primary template `coroutine_traits` such that if `ArgTypes` is a parameter pack of types and if the *qualified-id* `R::promise_type` is valid and denotes a type (13.10.2), then `coroutine_traits<R,ArgTypes...>` has the following publicly accessible member:

```
    using promise_type = typename R::promise_type;
```

Otherwise, `coroutine_traits<R,ArgTypes...>` has no members.

20.5.2 Header `<tuple>` synopsis [tuple.syn]

```
#include <compare>          // see 17.11.1

namespace std {
  // ...

  // 20.5.6, tuple helper classes
  template<class T> struct tuple_size;               // not defined
  template<class T> struct tuple_size<const T>;

  template<class... Types> struct tuple_size<tuple<Types...>>;

  template<size_t I, class T> struct tuple_element; // not defined
  template<size_t I, class T> struct tuple_element<I, const T>;

  template<size_t I, class... Types>
    struct tuple_element<I, tuple<Types...>>;
  template<size_t I, class T>
    using tuple_element_t = typename tuple_element<I, T>::type;

  // ...
}
```

20.7.2 Header `<variant>` synopsis [variant.syn]

```
#include <compare>          // see 17.11.1

namespace std {
  // 20.7.3, class template variant
  template<class... Types>
    class variant;

  // 20.7.4, variant helper classes
  template<class T> struct variant_size;                        // not defined
  template<class T> struct variant_size<const T>;
  template<class T>
    inline constexpr size_t variant_size_v = variant_size<T>::value;

  template<class... Types>
    struct variant_size<variant<Types...>>;

  template<size_t I, class T> struct variant_alternative;       // not defined
  template<size_t I, class T> struct variant_alternative<I, const T>;
  template<size_t I, class T>
    using variant_alternative_t = typename variant_alternative<I, T>::type;

  template<size_t I, class... Types>
    struct variant_alternative<I, variant<Types...>>;

  inline constexpr size_t variant_npos = -1;

  // ...
}
```

20.9.2 Class template `bitset` [template.bitset]

```
namespace std {
  template<size_t N> class bitset {
  public:
    // elided declarations ...

    // 20.9.2.1, constructors
    constexpr bitset() noexcept;
    constexpr bitset(unsigned long long val) noexcept;
    template<class charT, class traits, class Allocator>
      explicit bitset(
        const basic_string<charT, traits, Allocator>& str,
        typename basic_string<charT, traits, Allocator>::size_type pos = 0,
        typename basic_string<charT, traits, Allocator>::size_type n
          = basic_string<charT, traits, Allocator>::npos,
        charT zero = charT('0'),
        charT one = charT('1'));
```

```
    template <class charT>
      explicit bitset(
        const charT* str,
        typename basic_string<charT>::size_type n = basic_string<charT>::npos,
        charT zero = charT('0'),
        charT one = charT('1'));

    // more elided declarations ...
  };
}
```

20.9.2.1 Constructors [bitset.cons]

```
template<class charT, class traits, class Allocator>
  explicit bitset(
    const basic_string<charT, traits, Allocator>& str,
    typename basic_string<charT, traits, Allocator>::size_type pos = 0,
    typename basic_string<charT, traits, Allocator>::size_type n
      = basic_string<charT, traits, Allocator>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
```

3. *Effects:* Determines the effective length ...

```
template <class charT>
  explicit bitset(
    const charT* str,
    typename basic_string<charT>::size_type n = basic_string<charT>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
```

8. *Effects:* As if by:

```
     bitset(n == basic_string<charT>::npos
               ? basic_string<charT>(str)
               : basic_string<charT>(str, n),
            0, n, zero, one)
```

20.10.2 Header `<memory>` synopsis [memory.syn]

```
namespace std {
// ...

// 20.11.1, class template unique_ptr
// ...

template<class T1, class D1, class T2, class D2>
  bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
  bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
  bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
  bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
  requires three_way_comparable_with<typename unique_ptr<T1, D1>::pointer,
                                     typename unique_ptr<T2, D2>::pointer>
  compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer,
                             typename unique_ptr<T2, D2>::pointer>
    operator<=>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

template<class T, class D>
  bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D>
  bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
  bool operator<(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
  bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
  bool operator>(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
  bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
  bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
```

```
template<class T, class D>
  bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
  bool operator>=(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
  requires three_way_comparable_with<typename unique_ptr<T, D>::pointer, nullptr_t>
  compare_three_way_result_t<typename unique_ptr<T, D>::pointer, nullptr_t>
    operator<=>(const unique_ptr<T, D>& x, nullptr_t);

// ...
}
```

20.10.9 Allocator traits [allocator.traits]

1. The class template `allocator_traits` supplies ...

```
namespace std {
  template<class Alloc> struct allocator_traits {
    using allocator_type     = Alloc;

    using value_type         = typename Alloc::value_type;

    using pointer            = see below
    using const_pointer      = see below
    using void_pointer       = see below
    using const_void_pointer = see below

    using difference_type    = see below
    using size_type          = see below

    // ...
```

20.11.1.5 Specialized algorithms [unique.ptr.special]

```
template <class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

4. Let `CT` denote

```
common_type_t<typename unique_ptr<T1, D1>::pointer,
              typename unique_ptr<T2, D2>::pointer>
```

5. *Mandates:*
    1. — `unique_ptr<T1, D1>::pointer` is implicitly convertible to `CT` and
    2. — `unique_ptr<T2, D2>::pointer` is implicitly convertible to `CT`.

> Note the existing documentation convention to frequently omit `typename` in normative text that names a type in descriptive text.

20.11.3.9 Casts [util.smartptr.shared.cast]

```
templatecl<ass T, class U>
  shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
templatecl<ass T, class U>
  shared_ptr<T> static_pointer_cast(shared_ptr<U>&& r) noexcept;
```

1. *Mandates:* The expression `static_cast<T*>((U*)nullptr)` shall be well-formed.
2. *Returns:*

```
shared_ptr<T>(R, static_cast<typename shared_ptr<T>::element_type*>(r.get()))
```

   where *R* is `r` for the first overload, and `std::move(r)` for the second.
3. [*Note:* The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template <class T, class U>
  shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template <class T, class U>
  shared_ptr<T> dynamic_pointer_cast(shared_ptr<U>&& r) noexcept;
```

4. *Mandates:* The expression `dynamic_cast<T*>((U*)nullptr)` is well-formed. The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` is well formed.
5. *Preconditions:* The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` has well-defined behavior.

6. *Returns:*
   1. When `dynamic_cast<~~typename~~ shared_ptr<T>::element_type*>(r.get())` returns a non-null value `p`, `shared_ptr<T>(R, p)`, where *R* is `r` for the first overload, and `std::move(r)` for the second.
   2. Otherwise, `shared_ptr<T>()`.
7. [*Note:* The seemingly equivalent expression `shared_ptr<T>(dynamic_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template <class T, class U>
  shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template <class T, class U>
  shared_ptr<T> const_pointer_cast(shared_ptr<U>&& r) noexcept;
```

8. *Mandates:* The expression `const_cast<T*>((U*)nullptr)` shall be well-formed.
9. *Returns:*

```
shared_ptr<T>(R, const_cast<~~typename~~ shared_ptr<T>::element_type*>(r.get()))
```

   where *R* is `r` for the first overload, and `std::move(r)` for the second.
10. [*Note:* The seemingly equivalent expression `shared_ptr<T>(const_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template <class T, class U>
  shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template <class T, class U>
  shared_ptr<T> reinterpret_pointer_cast(shared_ptr<U>&& r) noexcept;
```

11. *Mandates:* The expression `reinterpret_cast<T*>((U*)nullptr)` shall be well-formed.
12. *Returns:*

```
shared_ptr<T>(R, reinterpret_cast<~~typename~~ shared_ptr<T>::element_type*>(r.get()))
```

   where *R* is `r` for the first overload, and `std::move(r)` for the second.
13. [*Note:* The seemingly equivalent expression `shared_ptr<T>(reinterpret_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

20.11.7 Smart pointer hash support [util.smartptr.hash]

```
template<class T, class D> struct hash<unique_ptr<T, D>>;
```

1. Letting `UP` be `unique_ptr<T,D>`, the specialization `hash<UP>` is enabled (20.14.18) if and only if `hash<typename UP::pointer>` is enabled. When enabled, for an object `p` of type `UP`, `hash<UP>()(p)` evaluates to the same value as `hash<typename UP::pointer>()(p.get())`. The member functions are not guaranteed to be `noexcept`.

```
template<class T> struct hash<shared_ptr<T>>;
```

2. For an object `p` of type `shared_ptr<T>`, `hash<shared_ptr<T>>()(p)` evaluates to the same value as `hash<typename shared_ptr<T>::element_type*>()(p.get())`.

20.13.1 Header `<scoped_allocator>` synopsis [allocator.adaptor.syn]

```
namespace std {
// class template scoped allocator adaptor
template<class OuterAlloc, class... InnerAlloc>
  class scoped_allocator_adaptor;

// 20.13.5, scoped allocator operators
template<class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
}
```

1. The class template `scoped_allocator_adaptor` is ...

```
namespace std {
  template<class OuterAlloc, class... InnerAlloc>
    class scoped_allocator_adaptor : public OuterAlloc {
    private:
      using OuterTraits = allocator_traits<OuterAlloc>;  // exposition only
      scoped_allocator_adaptor<InnerAllocs...> inner;    // exposition only

    public:
      using outer_allocator_type = OuterAlloc;
      using inner_allocator_type = see below;
```

```
      using value_type           = typename OuterTraits::value_type;
      using size_type            = typename OuterTraits::size_type;
      using difference_type      = typename OuterTraits::difference_type;
      using pointer              = typename OuterTraits::pointer;
      using const_pointer        = typename OuterTraits::const_pointer;
      using void_pointer         = typename OuterTraits::void_pointer;
      using const_void_pointer   = typename OuterTraits::const_void_pointer;

      // ...
  };
}
```

23.14.1 Header `<functional>` synopsis [functional.syn]

```
namespace std {
// ...

// 20.14.17, searchers
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
  class default_searcher;
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
  class boyer_moore_searcher;
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
  class boyer_moore_horspool_searcher;

// ...
}
```

20.14.17.2 Class template `boyer_moore_searcher` [func.search.bm]

```
template<class RandomAccessIterator1,
         class Hash = hash<typename iterator_traits<RandomAccessIterator1>::value_type>,
         class BinaryPredicate = equal_to<>>
  class boyer_moore_searcher {
  // ...
};
```

20.14.17.3 Class template `boyer_moore_horspool_searcher` [func.search.bmh]

```
template<class RandomAccessIterator1,
         class Hash = hash<typename iterator_traits<RandomAccessIterator1>::value_type>,
         class BinaryPredicate = equal_to<>>
  class boyer_moore_horspool_searcher {
  // ...
};
```

20.15.2 Header `<type_traits>` synopsis [meta.type.synop]

```
namespace std {
// 20.15.3, helper class
// ...

// 20.15.7.1, const-volatile modifications
template<class T> struct remove_const;
template<class T> struct remove_volatile;
template<class T> struct remove_cv;
template<class T> struct add_const;
template<class T> struct add_volatile;
template<class T> struct add_cv;

template<class T>
  using remove_const_t    = typename remove_const<T>::type;
template<class T>
  using remove_volatile_t = typename remove_volatile<T>::type;
template<class T>
  using remove_cv_t       = typename remove_cv<T>::type;
template<class T>
  using add_const_t       = typename add_const<T>::type;
template<class T>
  using add_volatile_t    = typename add_volatile<T>::type;
template<class T>
  using add_cv_t          = typename add_cv<T>::type;
```

```
// 20.15.7.2, reference modifications
template<class T> struct remove_reference;
template<class T> struct add_lvalue_reference;
template<class T> struct add_rvalue_reference;

template<class T>
  using remove_reference_t     = typename remove_reference<T>::type;
template<class T>
  using add_lvalue_reference_t = typename add_lvalue_reference<T>::type;
template<class T>
  using add_rvalue_reference_t = typename add_rvalue_reference<T>::type;

// 20.15.7.3, sign modifications
template<class T> struct make_signed;
template<class T> struct make_unsigned;

template<class T>
  using make_signed_t   = typename make_signed<T>::type;
template<class T>
  using make_unsigned_t = typename make_unsigned<T>::type;

// 20.15.7.4, array modifications
 template<class T> struct remove_extent;
template<class T> struct remove_all_extents;

template<class T>
  using remove_extent_t      = typename remove_extent<T>::type;
template<class T>
  using remove_all_extents_t = typename remove_all_extents<T>::type;

// 20.15.7.5, pointer modifications
 template<class T> struct remove_pointer;
 template<class T> struct add_pointer;

template<class T>
  using remove_pointer_t = typename remove_pointer<T>::type;
template<class T>
  using add_pointer_t    = typename add_pointer<T>::type;

// 20.15.7.6, other transformations
template<class T> struct type_identity;
template<size_t Len, size_t Align = default-alignment> // see 20.15.7.6
  struct aligned_storage;
template<size_t Len, class... Types> struct aligned_union;
template<class T> struct remove_cvref;
template<class T> struct decay;
template<bool, class T = void> struct enable_if;
template<bool, class T, class F> struct conditional;
template<class... T> struct common_type;
template<class T, class U, template<class> class TQual, template<class> class UQual>
  struct basic_common_reference { };
template<class... T> struct common_reference;
template<class T> struct underlying_type;
template<class Fn, class... ArgTypes> struct invoke_result;
template<class T> struct unwrap_reference;
template<class T> struct unwrap_ref_decay;

template<class T>
  using type_identity_t    = typename type_identity<T>::type;
template<size_t Len, size_t Align = default-alignment> // see 20.15.7.6
  using aligned_storage_t = typename aligned_storage<Len, Align>::type;
template<size_t Len, class... Types>
  using aligned_union_t = typename aligned_union<Len, Types...>::type;
template<class T>
  using remove_cvref_t = typename remove_cvref<T>::type;
template<class T>
   using decay_t   = typename decay<T>::type;
template<bool b, class T = void>
  using enable_if_t        = typename enable_if<b, T>::type;
template<bool b, class T, class F>
  using conditional_t      = typename conditional<b, T, F>::type;
template<class... T>
  using common_type_t      = typename common_type<T...>::type;
template<class... T>
  using common_reference_t = typename common_reference<T...>::type;
template<class T>
  using underlying_type_t  = typename underlying_type<T>::type;
template<class Fn, class... ArgTypes>
  using invoke_result_t    = typename invoke_result<Fn, ArgTypes...>::type;
```

```
template<class T>
  using unwrap_reference_t = typename unwrap_reference<T>::type;
template<class T>
  using unwrap_ref_decay_t = typename unwrap_ref_decay<T>::type;
template<class...>
  using void_t             = void;

// 20.15.8, logical operator traits
// ...

}
```

20.20.5.3 Class template `basic_format_parse_context` [format.parse.ctx]

```
namespace std {
  template<class charT>
  class basic_format_parse_context {
  public:
    using char_type = charT;
    using const_iterator = typename basic_string_view<charT>::const_iterator;
    using iterator = const_iterator;

    // ...
  };
}
```

20.20.6.1 Class template `basic_format_arg` [format.arg]

```
namespace std {
  template<class Context>
  class basic_format_arg {
  public:
    class handle;

  private:
    using char_type = typename Context::char_type;

    // ...
  };
}

template<class T> explicit basic_format_arg(const T& v) noexcept;
```

4. *Constraints:* The template specialization

   ```
   typename Context::template formatter_type<T>
   ```

   meets the Formatter requirements (20.20.5.1). The extent to which an implementation determines that the specialization meets the Formatter requirements is unspecified, except that as a minimum the expression

   ```
   typename Context::template formatter_type<T>()
     .format(declval<const T&>(), declval<Context&>())
   ```

   shall be well-formed when treated as an unevaluated operand.

```
template<class T> explicit handle(const T& val) noexcept;
```

17. *Effects:* Initializes `ptr_` with `addressof(val)` and `format_` with

```
[](basic_format_parse_context<char_type>& parse_ctx,
   Context& format_ctx, const void* ptr) {
  typename Context::template formatter_type<T> f;
  parse_ctx.advance_to(f.parse(parse_ctx));
  format_ctx.advance_to(f.format(*static_cast<const T*>(ptr), format_ctx));
}
```

20.20.6.2 Class template *`format-arg-store`* [format.arg.store]

```
template<class Context = format_context, class... Args>
format-arg-store<Context, Args...> make_format_args(const Args&... args);
```

2. *Preconditions:* : The type `typename Context::template formatter_type<`T_i`>` meets the *Formatter* requirements (20.20.5.1) for each T_i in `Args`.
3. *Returns:* `{basic_format_arg<Context>(args)...}`.

21.3.1 Header `<string>` synopsis [string.syn]

```
namespace std {

  // ...

  // 21.3.3.5, erasure
  template<class charT, class traits, class Allocator, class U>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
      erase(basic_string<charT, traits, Allocator>& c, const U& value);
  template<class charT, class traits, class Allocator, class Predicate>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
      erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);

  // basic_string typedef names
  // ...
}
```

21.3.2 Class template `basic_string` [basic.string]

```
namespace std {
  template<class charT, class traits = char_traits<charT>,
           class Allocator = allocator<charT>>
  class basic_string {
  public:
    // types
    using traits_type            = traits;
    using value_type             = charT;
    using allocator_type         = Allocator;
    using size_type              = typename allocator_traits<Allocator>::size_type;
    using difference_type        = typename allocator_traits<Allocator>::difference_type;
    using pointer                = typename allocator_traits<Allocator>::pointer;
    using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
    using reference              = value_type&;
    using const_reference        = const value_type&;

    using iterator               = implementation-defined; // see 22.2
    using const_iterator         = implementation-defined       ; // see 22.2
    using reverse_iterator       = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::reverse_iterator<const_iterator>;
    static const size_type npos  = -1;

    // 21.3.2.2, construct/copy/destroy
    // ...
  };


  template<class InputIterator,
           class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
    basic_string(InputIterator, InputIterator, Allocator = Allocator())
      -> basic_string<typename iterator_traits<InputIterator>::value_type,
                      char_traits<typename iterator_traits<InputIterator>::value_type>,
                      Allocator>;
  template<class charT,
           class traits,
           class Allocator = allocator<charT>>
    explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator())
      -> basic_string<charT, traits, Allocator>;

  template<class charT,
           class traits,
           class Allocator = allocator<charT>>
    basic_string(basic_string_view<charT, traits>,
                 typename see below::size_type, typename see below::size_type,
                 const Allocator& = Allocator())
      -> basic_string<charT, traits, Allocator>;
}
```

21.3.3.5 Erasure [string.erasure]

```
template<class charT, class traits, class Allocator, class U>
  constexpr typename basic_string<charT, traits, Allocator>::size_type
    erase(basic_string<charT, traits, Allocator>& c, const U& value);
```

1. *Effects:* Equivalent to:

```
 auto it = remove(c.begin(), c.end(), value);
 auto r = distance(it, c.end());
```

```
   c.erase(it, c.end());
   return r;

template<class charT, class traits, class Allocator, class Predicate>
  constexpr typename basic_string<charT, traits, Allocator>::size_type
    erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);
```

2. *Effects:* Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

22.2.4.1 Overview [container.node.overview]

```
template<unspecified>
class node-handle {
  // ...

private:
  using container_node_type = unspecified;
  using ator_traits = allocator_traits<allocator_type>;

  typename ator_traits::rebind_traits<container_node_type>::pointer ptr_;
  optional<allocator_type> alloc_;

  // ...
};
```

22.3.1 In general [sequences.general]

1. The headers `<array>` (22.3.2), `<deque>` (22.3.3), `<forward_list>` (22.3.4), `<list>` (22.3.5), and `<vector>` (22.3.6) define class templates that meet the requirements for sequence containers.
2. The following exposition-only alias template may appear in deduction guides for sequence containers:

```
  template<class InputIterator>
    using iter-value-type = typename iterator_traits<InputIterator>::value_type;  // exposition only
```

22.3.3 Header `<deque>` synopsis [deque.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1
namespace std {
  // 22.3.8, class template deque
  template<class T, class Allocator = allocator<T>> class deque;

  template<class T, class Allocator>
    bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template<class T, class Allocator>
    synth-three-way-result<T> operator<=>(const deque<T, Allocator>& x,
                                          const deque<T, Allocator>& y);

  template<class T, class Allocator>
    void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class T, class Allocator, class U>
    typename deque<T, Allocator>::size_type
      erase(deque<T, Allocator>& c, const U& value);
  template<class T, class Allocator, class Predicate>
    typename deque<T, Allocator>::size_type
      erase_if(deque<T, Allocator>& c, Predicate pred);

  namespace pmr {
    template<class T>
      using deque = std::deque<T, polymorphic_allocator<T>>;
  }
}
```

22.3.4 Header `<forward_list>` synopsis [forward.list.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1
namespace std {
  // 22.3.9, class template forward_list
```

```
  template<class T, class Allocator = allocator<T>> class forward_list;

  template<class T, class Allocator>
    bool operator==(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template<class T, class Allocator>
    synth-three-way-result<T> operator<=>(const forward_list<T, Allocator>& x,
                                          const forward_list<T, Allocator>& y);

  template<class T, class Allocator>
    void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class T, class Allocator, class U>
    typename forward_list<T, Allocator>::size_type
      erase(forward_list<T, Allocator>& c, const U& value);
  template<class T, class Allocator, class Predicate>
    typename forward_list<T, Allocator>::size_type
      erase_if(forward_list<T, Allocator>& c, Predicate pred);

  namespace pmr {
    template<class T>
      using forward_list = std::forward_list<T, polymorphic_allocator<T>>;
  }
}
```

22.3.5 Header `<list>` synopsis [list.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1
namespace std {
  // 22.3.10, class template list
  template<class T, class Allocator = allocator<T>> class list;

  template<class T, class Allocator>
    bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template<class T, class Allocator>
    synth-three-way-result<T> operator<=>(const list<T, Allocator>& x,
                                          const list<T, Allocator>& y);

  template<class T, class Allocator>
    void swap(list<T, Allocator>& x, list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class T, class Allocator, class U>
    typename list<T, Allocator>::size_type
      erase(list<T, Allocator>& c, const U& value);
  template<class T, class Allocator, class Predicate>
    typename list<T, Allocator>::size_type
      erase_if(list<T, Allocator>& c, Predicate pred);

  namespace pmr {
    template<class T>
      using list = std::list<T, polymorphic_allocator<T>>;
  }
}
```

22.3.6 Header `<vector>` synopsis [vector.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1
namespace std {
  // 22.3.11, class template vector
  template<class T, class Allocator = allocator<T>> class vector;

  template<class T, class Allocator>
    bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template<class T, class Allocator>
    synth-three-way-result<T> operator<=>(const vector<T, Allocator>& x,
                                          const vector<T, Allocator>& y);

  template<class T, class Allocator>
    void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class T, class Allocator, class U>
    typename vector<T, Allocator>::size_type
      erase(vector<T, Allocator>& c, const U& value);
  template<class T, class Allocator, class Predicate>
```

```
    typename vector<T, Allocator>::size_type
      erase_if(vector<T, Allocator>& c, Predicate pred);

  // 22.3.12, class vector<bool>
  template<class Allocator> class vector<bool, Allocator>;

  // hash support
  template<class T> struct hash;
  template<class Allocator> struct hash<vector<bool, Allocator>>;

  namespace pmr {
    template<class T>
      using vector = std::vector<T, polymorphic_allocator<T>>;
  }
}
```

22.3.8.1 Overview [deque.overview]

2. A `deque` meets all of the requirements of a container, ...

```
namespace std {
  template<class T, class Allocator = allocator<T>>
    class deque {
    public:
      // types
      using value_type             = T;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2
      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using reverse_iterator       = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;

      // ...
    };
}
```

22.3.8.5 Erasure [deque.erasure]

```
template<class T, class Allocator, class U>
  typename deque<T, Allocator>::size_type
    erase(deque<T, Allocator>& c, const U& value);
```

1. *Effects:* Equivalent to:

```
   auto it = remove(c.begin(), c.end(), value);
   auto r = distance(it, c.end());
   c.erase(it, c.end());
   return r;
```

```
template<class T, class Allocator, class Predicate>
  typename deque<T, Allocator>::size_type
    erase_if(deque<T, Allocator>& c, Predicate pred);
```

2. *Effects:* Equivalent to:

```
   auto it = remove_if(c.begin(), c.end(), pred);
   auto r = distance(it, c.end());
   c.erase(it, c.end());
   return r;
```

22.3.9.1 Overview [forwardlist.overview]

2. A `forward_list` meets all of the requirements of a container, ...

```
namespace std {
  template<class T, class Allocator = allocator<T>>
    class forward_list {
    public:
      // types
      using value_type             = T;
      using allocator_type         = Allocator;
```

```
    using pointer               = typename allocator_traits<Allocator>::pointer;
    using const_pointer         = typename allocator_traits<Allocator>::const_pointer;
    using reference             = value_type&;
    using const_reference       = const value_type&;
    using size_type             = implementation-defined; // see 22.2
    using difference_type       = implementation-defined; // see 22.2
    using iterator              = implementation-defined; // see 22.2
    using const_iterator        = implementation-defined; // see 22.2

    // ...
  };
}
```

22.3.9.7 Erasure [forward.list.erasure]

```
template<class T, class Allocator, class U>
  typename forward_list<T, Allocator>::size_type
    erase(forward_list<T, Allocator>& c, const U& value);
```

1. *Effects:* Equivalent to: `return erase_if(c, [&](auto& elem) { return elem == value; });`

```
template<class T, class Allocator, class Predicate>
  typename forward_list<T, Allocator>::size_type
    erase_if(forward_list<T, Allocator>& c, Predicate pred);
```

2. *Effects:* Equivalent to: `return c.remove_if(pred);`

22.3.10.1 Overview [list.overview]

2. A `list` meets all of the requirements of a container, ...

```
namespace std {
  template<class T, class Allocator = allocator<T>>
    class list {
    public:
      // types
      using value_type            = T;
      using allocator_type        = Allocator;
      using pointer               = typename allocator_traits<Allocator>::pointer;
      using const_pointer         = typename allocator_traits<Allocator>::const_pointer;
      using reference             = value_type&;
      using const_reference       = const value_type&;
      using size_type             = implementation-defined; // see 22.2
      using difference_type       = implementation-defined; // see 22.2
      using iterator              = implementation-defined; // see 22.2
      using const_iterator        = implementation-defined; // see 22.2
      using reverse_iterator      = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;

      // ...
    };
}
```

22.3.10.6 Erasure [list.erasure]

```
template<class T, class Allocator, class U>
  typename list<T, Allocator>::size_type
    erase(list<T, Allocator>& c, const U& value);
```

1. *Effects:* Equivalent to: `return erase_if(c, [&](auto& elem) { return elem == value; });`

```
template<class T, class Allocator, class Predicate>
  typename list<T, Allocator>::size_type
    erase_if(list<T, Allocator>& c, Predicate pred);
```

2. *Effects:* Equivalent to: `return c.remove_if(pred);`

22.3.11.1 Overview [vector.overview]

2. A `vector` meets all of the requirements of a container, ...

```
namespace std {
  template<class T, class Allocator = allocator<T>>
    class vector {
    public:
      // types
```

```
    using value_type             = T;
    using allocator_type         = Allocator;
    using pointer                = typename allocator_traits<Allocator>::pointer;
    using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
    using reference              = value_type&;
    using const_reference        = const value_type&;
    using size_type              = implementation-defined; // see 22.2
    using difference_type        = implementation-defined; // see 22.2
    using iterator               = implementation-defined; // see 22.2
    using const_iterator         = implementation-defined; // see 22.2
    using reverse_iterator       = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::reverse_iterator<const_iterator>;

    // ...
  };
}
```

22.3.11.6 Erasure [vector.erasure]

```
template<class T, class Allocator, class U>
  typename vector<T, Allocator>::size_type
    erase(vector<T, Allocator>& c, const U& value);
```

1. *Effects:* Equivalent to:

```
  auto it = remove(c.begin(), c.end(), value);
  auto r = distance(it, c.end());
  c.erase(it, c.end());
  return r;
```

```
template<class T, class Allocator, class Predicate>
  typename vector<T, Allocator>::size_type
    erase_if(vector<T, Allocator>& c, Predicate pred);
```

2. *Effects:* Equivalent to:

```
  auto it = remove_if(c.begin(), c.end(), pred);
  auto r = distance(it, c.end());
  c.erase(it, c.end());
  return r;
```

22.4.1 In general [associative.general]

2. The following exposition-only alias templates may appear in deduction guides for associative containers:

```
  template<class InputIterator>
    using iter-value-type =
      typename iterator_traits<InputIterator>::value_type;               // exposition only
  template<class InputIterator>
    using iter-key-type = remove_const_t<
      typename iterator_traits<InputIterator>::value_type::first_type>;  // exposition only
  template<class InputIterator>
    using iter-mapped-type =
      typename iterator_traits<InputIterator>::value_type::second_type;  // exposition only
  template<class InputIterator>
    using iter-to-alloc-type = pair<
      add_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
      typename iterator_traits<InputIterator>::value_type::second_type>; // exposition only
```

22.4.2 Header <map> synopsis [associative.map.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1

namespace std {
  // 22.4.4, class template map
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class map;

  template<class Key, class T, class Compare, class Allocator>
    bool operator==(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template<class Key, class T, class Compare, class Allocator>
    synth-three-way-result<T> operator<=>(const map<Key, T, Compare, Allocator>& x,
                                          const map<Key, T, Compare, Allocator>& y);
```

```
  template<class Key, class T, class Compare, class Allocator>
    void swap(map<Key, T, Compare, Allocator>& x,
              map<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class Key, class T, class Compare, class Allocator, class Predicate>
    typename map<Key, T, Compare, Allocator>::size_type
      erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);

  // 22.4.5, class template multimap
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class multimap;

  template<class Key, class T, class Compare, class Allocator>
    bool operator==(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template<class Key, class T, class Compare, class Allocator>
    synth-three-way-result<T> operator<=>(const multimap<Key, T, Compare, Allocator>& x,
                                          const multimap<Key, T, Compare, Allocator>& y);

  template<class Key, class T, class Compare, class Allocator>
    void swap(multimap<Key, T, Compare, Allocator>& x,
              multimap<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class Key, class T, class Compare, class Allocator, class Predicate>
    typename multimap<Key, T, Compare, Allocator>::size_type
      erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);

  namespace pmr {
    template<class Key, class T, class Compare = less<Key>>
      using map = std::map<Key, T, Compare, polymorphic_allocator<T>>;

    template<class Key, class T, class Compare = less<Key>>
      using multimap = std::multimap<Key, T, Compare, polymorphic_allocator<T>>;
  }
}
```

22.4.3 Header <set> synopsis [associative.set.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1

namespace std {
  // 22.4.6, class template set
  template<class Key, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>
    class set;

  template<class Key, class Compare, class Allocator>
    bool operator==(const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
  template<class Key, class Compare, class Allocator>
    synth-three-way-result<T> operator<=>(const set<Key, Compare, Allocator>& x,
                                          const set<Key, Compare, Allocator>& y);

  template<class Key, class Compare, class Allocator>
    void swap(set<Key, Compare, Allocator>& x,
              set<Key, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template<class Key, class Compare, class Allocator, class Predicate>
    typename set<Key, Compare, Allocator>::size_type
      erase_if(set<Key, Compare, Allocator>& c, Predicate pred);

  // 22.4.7, class template multiset
  template<class Key, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>
    class multiset;

  template<class Key, class Compare, class Allocator>
    bool operator==(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template<class Key, class Compare, class Allocator>
    synth-three-way-result<T> operator<=>(const multiset<Key, Compare, Allocator>& x,
                                          const multiset<Key, Compare, Allocator>& y);

  template<class Key, class Compare, class Allocator>
    void swap(multiset<Key, Compare, Allocator>& x,
              multiset<Key, Compare, Allocator>& y)
```

```
      noexcept(noexcept(x.swap(y)));

  template<class Key, class Compare, class Allocator, class Predicate>
    typename multiset<Key, Compare, Allocator>::size_type
      erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);

  namespace pmr {
    template<class Key, class Compare = less<Key>>
      using set = std::set<Key, Compare, polymorphic_allocator<T>>;

    template<class Key, class Compare = less<Key>>
      using multiset = std::multiset<Key, Compare, polymorphic_allocator<T>>;
  }
}
```

22.4.4.1 Overview [map.overview]

2. A `map` meets all of the requirements of a container, ...

```
namespace std {
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class map {
    public:
      // types
      using key_type               = Key;
      using mapped_type            = T;
      using value_type             = Key;
      using key_compare            = pair<const Key, T>;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2
      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using reverse_iterator       = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;
      using node_type              = unspecified;
      using insert_return_type     = insert-return-type<iterator, node_type>;

      // ...
    };
}
```

22.4.4.5 Erasure [map.erasure]

```
template<class Key, class T, class Compare, class Allocator, class Predicate>
  typename map<Key, T, Compare, Allocator>::size_type
    erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
    auto original_size = c.size();
    for (auto i = c.begin(), last = c.end(); i != last; ) {
      if (pred(*i)) {
        i = c.erase(i);
      } else {
         ++i;
      }
    }
    return original_size - c.size();
```

22.4.5.1 Overview [multimap.overview]

2. A `multimap` meets all of the requirements of a container, ...

```
namespace std {
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class multimap {
    public:
      // types
      using key_type               = Key;
      using mapped_type            = T;
```

```
    using value_type             = Key;
    using key_compare            = pair<const Key, T>;
    using allocator_type         = Allocator;
    using pointer                = typename allocator_traits<Allocator>::pointer;
    using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
    using reference              = value_type&;
    using const_reference        = const value_type&;
    using size_type              = implementation-defined; // see 22.2
    using difference_type        = implementation-defined; // see 22.2
    using iterator               = implementation-defined; // see 22.2
    using const_iterator         = implementation-defined; // see 22.2
    using reverse_iterator       = std::reverse_iterator<iterator>;
    using const_reverse_iterator = std::reverse_iterator<const_iterator>;
    using node_type              = unspecified;

    // ...
  };
}
```

22.4.5.4 Erasure [multimap.erasure]

```
template<class Key, class T, class Compare, class Allocator, class Predicate>
  typename multimap<Key, T, Compare, Allocator>::size_type
    erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
  if (pred(*i)) {
    i = c.erase(i);
  } else {
     ++i;
  }
}
return original_size - c.size();
```

22.4.6.1 Overview [set.overview]

2. A `set` meets all of the requirements of a container, ...

```
namespace std {
  template<class Key, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>
    class set {
    public:
      // types
      using key_type             = Key;
      using key_compare          = Compare;
      using value_type           = Key;
      using value_compare        = Compare;
      using allocator_type       = Allocator;
      using pointer              = typename allocator_traits<Allocator>::pointer;
      using const_pointer        = typename allocator_traits<Allocator>::const_pointer;
      using reference            = value_type&;
      using const_reference      = const value_type&;
      using size_type            = implementation-defined; // see 22.2
      using difference_type      = implementation-defined; // see 22.2
      using iterator             = implementation-defined; // see 22.2
      using const_iterator       = implementation-defined; // see 22.2
      using reverse_iterator     = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;
      using node_type            = unspecified;
      using insert_return_type   = insert-return-type<iterator, node_type>;

      // ...
    };
}
```

22.4.6.3 Erasure [set.erasure]

```
template<class Key, class Compare, class Allocator, class Predicate>
  typename set<Key, Compare, Allocator>::size_type
    erase_if(set<Key, Compare, Allocator>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
    auto original_size = c.size();
    for (auto i = c.begin(), last = c.end(); i != last; ) {
      if (pred(*i)) {
        i = c.erase(i);
      } else {
         ++i;
      }
    }
    return original_size - c.size();
```

22.4.6.1 Overview [multiset.overview]

2. A `multiset` meets all of the requirements of a container, ...

```
namespace std {
  template<class Key, class Compare = less<Key>, class Allocator = allocator<pair<const Key, T>>
    class multiset {
    public:
      // types
      using key_type               = Key;
      using key_compare            = Compare;
      using value_type             = Key;
      using value_compare          = Compare;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2
      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using reverse_iterator       = std::reverse_iterator<iterator>;
      using const_reverse_iterator = std::reverse_iterator<const_iterator>;
      using node_type              = unspecified;

      // ...
    };
}
```

22.4.7.3 Erasure [multiset.erasure]

```
template<class Key, class Compare, class Allocator, class Predicate>
  typename multiset<Key, Compare, Allocator>::size_type
    erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
    auto original_size = c.size();
    for (auto i = c.begin(), last = c.end(); i != last; ) {
      if (pred(*i)) {
        i = c.erase(i);
      } else {
         ++i;
      }
    }
    return original_size - c.size();
```

22.5.2 Header `<unordered_map>` synopsis [unord.map.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1

namespace std {
  // ...

  template<class K, class T, class H, class P, class A, class Predicate>
    typename unordered_map<K, T, H, P, A>::size_type
      erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);
  template<class K, class T, class H, class P, class A, class Predicate>
    typename unordered_multimap<K, T, H, P, A>::size_type
      erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);

  // ...
}
```

22.5.3 Header `<unordered_set>` synopsis [unord.set.syn]

```
#include <compare>            // see 17.11.1
#include <initializer_list>   // see 17.10.1

namespace std {
  // ...

  template<class K, class H, class P, class A, class Predicate>
    typename unordered_set<K, H, P, A>::size_type
      erase_if(unordered_set<K, H, P, A>& c, Predicate pred);
  template<class K, class H, class P, class A, class Predicate>
    typename unordered_multiset<K, H, P, A>::size_type
      erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);

  // ...
}
```

22.5.4.1 Overview [unord.map.overview]

3. Subclause 22.5.4 only describes operations on `unordered_map` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_map {
    public:
      // types
      using key_type               = Key;
      using mapped_type            = T;
      using value_type             = pair<const Key, T>;;
      using hasher                 = Hash;
      using key_equal              = Pred;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2

      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using local_iterator         = implementation-defined; // see 22.2
      using const_local_iterator   = implementation-defined; // see 22.2
      using node_type              = unspecified;
      using insert_return_type     = insert-return-type<iterator, node_type>;

      // ...
  };

  template<class InputIterator,
           class Hash = hash<iter-key-type<InputIterator>>,
           class Pred = equal_to<iter-key-type<InputIterator>>,
           class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
    unordered_map(InputIterator, InputIterator, typename see below::size_type = see below,
                  Hash = Hash(), Pred = Pred(), Allocator = Allocator())
      -> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash, Pred,
                       Allocator>;

  template<class Key, class T, class Hash = hash<Key>,
           class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
    unordered_map(initializer_list<pair<const Key, T>>,
                  typename see below::size_type = see below, Hash = Hash(),
                  Pred = Pred(), Allocator = Allocator())
      -> unordered_map<Key, T, Hash, Pred, Allocator>;

  template<class InputIterator, class Allocator>
    unordered_map(InputIterator, InputIterator, typename see below::size_type, Allocator)
      -> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
                       hash<iter-key-type<InputIterator>>,
                       equal_to<iter-key-type<InputIterator>>, Allocator>;

  template<class InputIterator, class Allocator>
```

```
  unordered_map(InputIterator, InputIterator, Allocator)
    -> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
                     hash<iter-key-type<InputIterator>>,
                     equal_to<iter-key-type<InputIterator>>, Allocator>;

  template<class InputIterator, class Hash, class Allocator>
  unordered_map(InputIterator, InputIterator, typename see below::size_type, Hash, Allocator)
    -> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
                     equal_to<iter-key-type<InputIterator>>, Allocator>;

  template<class Key, class T, class Allocator>
  unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type,
                Allocator)
    -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

  template<class Key, class T, class Allocator>
  unordered_map(initializer_list<pair<const Key, T>>, Allocator)
    -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

  template<class Key, class T, class Hash, class Allocator>
  unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_type, Hash,
                Allocator)
    -> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;

  // ...
}
```

22.5.4.5 Erasure [unord.map.erasure]

```
template<class K, class T, class H, class P, class A, class Predicate>
  typename unordered_map<K, T, H, P, A>::size_type
    erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
  if (pred(*i)) {
    i = c.erase(i);
  } else {
     ++i;
  }
}
return original_size - c.size();
```

22.5.5.1 Overview [unord.multimap.overview]

3. Subclause 22.5.5 only describes operations on `unordered_multimap` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_multimap {
    public:
      // types
      using key_type              = Key;
      using mapped_type           = T;
      using value_type            = pair<const Key, T>;;
      using hasher                = Hash;
      using key_equal             = Pred;
      using allocator_type        = Allocator;
      using pointer               = typename allocator_traits<Allocator>::pointer;
      using const_pointer         = typename allocator_traits<Allocator>::const_pointer;
      using reference             = value_type&;
      using const_reference       = const value_type&;
      using size_type             = implementation-defined; // see 22.2
      using difference_type       = implementation-defined; // see 22.2

      using iterator              = implementation-defined; // see 22.2
      using const_iterator        = implementation-defined; // see 22.2
      using local_iterator        = implementation-defined; // see 22.2
      using const_local_iterator  = implementation-defined; // see 22.2
      using node_type             = unspecified;
```

```
    // ...
};

template<class InputIterator,
         class Hash = hash<iter-key-type<InputIterator>>,
         class Pred = equal_to<iter-key-type<InputIterator>>,
         class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
  unordered_multimap(InputIterator, InputIterator,
                     typename see below::size_type = see below,
                     Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash, Pred,
                          Allocator>;

template<class Key, class T, class Hash = hash<Key>,
         class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
  unordered_multimap(initializer_list<pair<const Key, T>>,
                     typename see below::size_type = see below,
                     Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_multimap<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
  unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Allocator)
    -> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
                          hash<iter-key-type<InputIterator>>,
                          equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
  unordered_multimap(InputIterator, InputIterator, Allocator)
    -> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
                          hash<iter-key-type<InputIterator>>,
                          equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
  unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Hash,
                     Allocator)
    -> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
                          equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class Key, class T, class Allocator>
  unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type,
                     Allocator)
    -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Allocator>
  unordered_multimap(initializer_list<pair<const Key, T>>, Allocator)
    -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
  unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_type,
                     Hash, Allocator)
    -> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;

  // ...
}
```

22.5.5.4 Erasure [unord.multimap.erasure]

```
template<class K, class T, class H, class P, class A, class Predicate>
  typename unordered_multimap<K, T, H, P, A>::size_type
    erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
  if (pred(*i)) {
    i = c.erase(i);
  } else {
     ++i;
  }
}
return original_size - c.size();
```

22.5.6.1 Overview [unord.set.overview]

3. Subclause 22.5.6 only describes operations on `unordered_set` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<Key>>
  class unordered_set {
    public:
      // types
      using key_type               = Key;
      using value_type             = Key;
      using hasher                 = Hash;
      using key_equal              = Pred;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2

      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using local_iterator         = implementation-defined; // see 22.2
      using const_local_iterator   = implementation-defined; // see 22.2
      using node_type              = unspecified;
      using insert_return_type     = insert-return-type<iterator, node_type>;

      // ...
  };

  template<class InputIterator,
           class Hash = hash<iter-value-type<InputIterator>>,
           class Pred = equal_to<iter-value-type<InputIterator>>,
           class Allocator = allocator<iter-value-type<InputIterator>>>
    unordered_set(InputIterator, InputIterator, typename see below::size_type = see below,
                  Hash = Hash(), Pred = Pred(), Allocator = Allocator())
      -> unordered_set<iter-value-type<InputIterator>, Hash, Pred, Allocator>;

  template<class T, class Hash = hash<T>,
           class Pred = equal_to<T>, class Allocator = allocator<T>>
    unordered_set(initializer_list<T>, typename see below::size_type = see below,
                  Hash = Hash(), Pred = Pred(), Allocator = Allocator())
      -> unordered_set<T, Hash, Pred, Allocator>;

  template<class InputIterator, class Allocator>
    unordered_set(InputIterator, InputIterator, typename see below::size_type, Allocator)
      -> unordered_set<iter-value-type<InputIterator>,
                       hash<iter-value-type<InputIterator>>,
                       equal_to<iter-value-type<InputIterator>>,
                       Allocator>;

  template<class InputIterator, class Hash, class Allocator>
    unordered_set(InputIterator, InputIterator, typename see below::size_type,
                  Hash, Allocator)
      -> unordered_set<iter-value-type<InputIterator>, Hash,
                       equal_to<iter-value-type<InputIterator>>,
                       Allocator>;

  template<class T, class Allocator>
    unordered_set(initializer_list<T>, typename see below::size_type, Allocator)
      -> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

  template<class T, class Hash, class Allocator>
    unordered_set(initializer_list<T>, typename see below::size_type, Hash, Allocator)
      -> unordered_set<T, Hash, equal_to<T>, Allocator>;

    // ...
}
```

22.5.6.3 Erasure [unord.set.erasure]

```
template<class K, class H, class P, class A, class Predicate>
  typename unordered_set<K, H, P, A>::size_type
    erase_if(unordered_set<K, H, P, A>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
   auto original_size = c.size();
   for (auto i = c.begin(), last = c.end(); i != last; ) {
```

```
    if (pred(*i)) {
      i = c.erase(i);
    } else {
       ++i;
    }
  }
  return original_size - c.size();
```

22.5.7.1 Overview [unord.multiset.overview]

3. Subclause 22.5.7 only describes operations on `unordered_multiset` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<Key>>
  class unordered_multiset {
    public:
      // types
      using key_type               = Key;
      using value_type             = Key;
      using hasher                 = Hash;
      using key_equal              = Pred;
      using allocator_type         = Allocator;
      using pointer                = typename allocator_traits<Allocator>::pointer;
      using const_pointer          = typename allocator_traits<Allocator>::const_pointer;
      using reference              = value_type&;
      using const_reference        = const value_type&;
      using size_type              = implementation-defined; // see 22.2
      using difference_type        = implementation-defined; // see 22.2

      using iterator               = implementation-defined; // see 22.2
      using const_iterator         = implementation-defined; // see 22.2
      using local_iterator         = implementation-defined; // see 22.2
      using const_local_iterator   = implementation-defined; // see 22.2
      using node_type              = unspecified;

      // ...

  };

  template<class InputIterator,
           class Hash = hash<iter-value-type<InputIterator>>,
           class Pred = equal_to<iter-value-type<InputIterator>>,
           class Allocator = allocator<iter-value-type<InputIterator>>>
    unordered_multiset(InputIterator, InputIterator, typename see below::size_type = see below,
                       Hash = Hash(), Pred = Pred(), Allocator = Allocator())
      -> unordered_multiset<iter-value-type<InputIterator>,
                            Hash, Pred, Allocator>;

  template<class T, class Hash = hash<T>,
           class Pred = equal_to<T>, class Allocator = allocator<T>>
    unordered_multiset(initializer_list<T>, typename see below::size_type = see below,
                       Hash = Hash(), Pred = Pred(), Allocator = Allocator())
      -> unordered_multiset<T, Hash, Pred, Allocator>;

  template<class InputIterator, class Allocator>
    unordered_multiset(InputIterator, InputIterator, typename see below::size_type, Allocator)
      -> unordered_multiset<iter-value-type<InputIterator>,
                            hash<iter-value-type<InputIterator>>,
                            equal_to<iter-value-type<InputIterator>>,
                            Allocator>;

  template<class InputIterator, class Hash, class Allocator>
    unordered_multiset(InputIterator, InputIterator, typename see below::size_type,
                       Hash, Allocator)
      -> unordered_multiset<iter-value-type<InputIterator>, Hash,
                            equal_to<iter-value-type<InputIterator>>,
                            Allocator>;

  template<class T, class Allocator>
    unordered_multiset(initializer_list<T>, typename see below::size_type, Allocator)
      -> unordered_multiset<T, hash<T>, equal_to<T>, Allocator>;

  template<class T, class Hash, class Allocator>
    unordered_multiset(initializer_list<T>, typename see below::size_type, Hash, Allocator)
      -> unordered_multiset<T, Hash, equal_to<T>, Allocator>;
```

```
    // ...
}
```

22.5.7.3 Erasure [unord.multiset.erasure]

```
template<class K, class H, class P, class A, class Predicate>
  typename unordered_multiset<K, H, P, A>::size_type
    erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);
```

1. *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
  if (pred(*i)) {
    i = c.erase(i);
  } else {
     ++i;
  }
}
return original_size - c.size();
```

22.6.2 Header <queue> synopsis [queue.syn]

```
#include <initializer_list>

namespace std {
  // ...

  template<class T, class Container = vector<T>,
           class Compare = less<typename Container::value_type>>
    class priority_queue;

  // ...
}
```

22.6.4.1 Definition [queue.defn]

1. Any sequence container supporting operations `front()`, `back()`, `push_back()` and `pop_front()` can be used to instantiate `queue`. In particular, `list` (26.3.10) and `deque` (26.3.8) can be used.

```
namespace std {
  template<class T, class Container = deque<T>>
  class queue {
   public:
     using value_type      = typename Container::value_type;
     using reference       = typename Container::reference;
     using const_reference = typename Container::const_reference;
     using size_type       = typename Container::size_type;
     using container_type  = Container;

  protected:
    Container c;

    // ...
  };

  template<class Container>
    queue(Container) -> queue<typename Container::value_type, Container>;

  template<class Container, class Allocator>
    queue(Container, Allocator) -> queue<typename Container::value_type, Container>;

  // ...
}
```

22.6.5.1 Overview [priqueue.overview]

1. Any sequence container with random access iterator and ...

```
namespace std {
  template<class T, class Container = vector<T>,
           class Compare = less<typename Container::value_type>>
   class priority_queue {
   public:
```

```
    using value_type      = typename Container::value_type;
    using reference       = typename typename Container::reference;
    using const_reference = typename typename Container::const_reference;
    using size_type       = typename typename Container::size_type;
    using container_type  = Container;
    using value_compare   = Compare;

  protected:
    Container c;
    Compare comp;

    // ...
  };

  template<class Compare, class Container>
    priority_queue(Compare, Container)
      -> priority_queue<typename Container::value_type, Container, Compare>;

  template<class InputIterator,
           class Compare = less<typename iterator_traits<InputIterator>::value_type>,
           class Container = vector<typename iterator_traits<InputIterator>::value_type>>
    priority_queue(InputIterator, InputIterator, Compare = Compare(), Container = Container())
      -> priority_queue<typename iterator_traits<InputIterator>::value_type, Container, Compare>;

  template<class Compare, class Container, class Allocator>
    priority_queue(Compare, Container, Allocator)
      -> priority_queue<typename Container::value_type, Container, Compare>;

  // ...
}
```

22.6.6.1 Definition [stack.defn]

```
namespace std {
  template<class T, class Container = deque<T>>
    class stack {
   public:
     using value_type      = typename Container::value_type;
     using reference       = typename typename Container::reference;
     using const_reference = typename typename Container::const_reference;
     using size_type       = typename typename Container::size_type;
     using container_type  = Container;

  protected:
    Container c;

    // ...
  };

  template<class Container>
    stack(Container) -> stack<typename Container::value_type, Container>;

  template<class Container, class Allocator>
    stack(Container, Allocator) -> stack<typename Container::value_type, Container>;

  // ...
}
```

23.2 Header `<iterator>` synopsis [iterator.synopsis]

```
#include <compare>
#include <concepts>

namespace std {
  // ...

  // 23.4.2, iterator operations
  template<class InputIterator, class Distance>
    constexpr void
      advance(InputIterator& i, Distance n);
  template<class InputIterator>
    constexpr typename iterator_traits<InputIterator>::difference_type
      distance(InputIterator first, InputIterator last);
  template<class InputIterator>
    constexpr InputIterator
      next(InputIterator x,
           typename iterator_traits<InputIterator>::difference_type n = 1);
  template<class BidirectionalIterator>
```

```
    constexpr BidirectionalIterator
      prev(BidirectionalIterator x,
           typename iterator_traits<BidirectionalIterator>::difference_type n = 1);

  // ...

  // 23.5, predefined iterators and sentinels

  // ...

  template<class Iterator1, class Iterator2>
    constexpr auto operator-(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
  template<class Iterator>
    constexpr reverse_iterator<Iterator> operator+(
      typename reverse_iterator<Iterator>::difference_type n, const reverse_iterator<Iterator>& x);

  template<class Iterator>
    constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i);

  // ...

  template<class Iterator1, class Iterator2>
    constexpr auto operator-(
    const move_iterator<Iterator1>& x,
    const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
  template<class Iterator>
    constexpr move_iterator<Iterator> operator+(
      typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);

  template<class Iterator>
    constexpr move_iterator<Iterator> make_move_iterator(Iterator i);

  ...

}
```

23.3.2.1 Incrementable traits [incrementable.traits]

1. To implement algorithms only in terms of incrementable types, it is often necessary to determine the difference type that corresponds to a particular incrementable type. Accordingly, it is required that if `WI` is the name of a type that models the `weakly_incrementable` concept (23.3.4.4), the type

   ```
   iter_difference_t<WI>
   ```

   be defined as the incrementable type's difference type.

```
namespace std {
  template<class> struct incrementable_traits { };

  template<class T>
    requires is_object_v<T>
  struct incrementable_traits<T*> {
    using difference_type = ptrdiff_t;
  };

  template<class I>
  struct incrementable_traits<const I>
    : incrementable_traits<I> { };

  template<class T>
    requires requires { typename T::difference_type; }
  struct incrementable_traits<T> {
    using difference_type = typename T::difference_type;
  };

  template<class T>
    requires (!requires { typename T::difference_type; } &&
              requires(const T& a, const T& b) { { a - b } -> integral; })
  struct incrementable_traits<T> {
    using difference_type = make_signed_t<decltype(declval<T>() - declval<T>())>;
  };

  template<class T>
    using iter_difference_t = see below;
}
```

23.3.2.3 Iterator traits [iterator.traits]

2. The definitions in this subclause make use of the following exposition-only concepts:

```
template<class I>
concept cpp17-iterator =
  copyable<I> && requires(I i) {
    {   *i } -> can-reference;
    {  ++i } -> same_as<I&>;
    { *i++ } -> can-reference;
  };

template<class I>
  concept cpp17-input-iterator =
    cpp17-iterator<I> && equality_comparable<I> && requires(I i) {
      typename incrementable_traits<I>::difference_type;
      typename indirectly_readable_traits<I>::value_type;
      typename common_reference_t<iter_reference_t<I>&&,
                                  typename indirectly_readable_traits<I>::value_type&>;
      typename common_reference_t<decltype(*i++)&&,
                                  typename indirectly_readable_traits<I>::value_type&>;
         requires signed_integral<typename incrementable_traits<I>::difference_type>;
  };

template<class I>
  concept cpp17-forward-iterator =
    cpp17-input-iterator<I> && constructible_from<I> &&
    is_lvalue_reference_v<iter_reference_t<I>> &&
    same_as<remove_cvref_t<iter_reference_t<I>>,
            typename indirectly_readable_traits<I>::value_type> &&
    requires(I i) {
      {  i++ } -> convertible_to<const I&>;
      { *i++ } -> same_as<iter_reference_t<I>>;
    };

template<class I>
  concept cpp17-bidirectional-iterator =
    cpp17-forward-iterator<I> && requires(I i) {
      {  --i } -> same_as<I&>;
      {  i-- } -> convertible_to<const I&>;
      { *i-- } -> same_as<iter_reference_t<I>>;
  };

template<class I>
  concept cpp17-random-access-iterator =
    cpp17-bidirectional-iterator<I> && totally_ordered<I> &&
    requires(I i, typename incrementable_traits<I>::difference_type n) {
      { i += n } -> same_as<I&>;
      { i -= n } -> same_as<I&>;
      { i +  n } -> same_as<I>;
      { n +  i } -> same_as<I>;
      { i -  n } -> same_as<I>;
      { i -  i } -> same_as<decltype(n)>;
      {  i[n]  } -> convertible_to<iter_reference_t<I>>;
  };
```

3. The members of a specialization `iterator_traits<I>` generated from the `iterator_traits` primary template are computed as follows:
   1. — If `I` has valid (13.10.2) member types `difference_type`, `value_type`, `reference`, and `iterator_category`, then `iterator_traits<I>` has the following publicly accessible members:

```
using iterator_category = typename I::iterator_category;
using value_type        = typename I::value_type;
using difference_type   = typename I::difference_type;
using pointer           = see below;
using reference         = typename I::reference;
```

      If the qualified-id `I::pointer` is valid and denotes a type, then `iterator_traits<I>::pointer` names that type; otherwise, it names `void`.
   2. — Otherwise, if `I` satisfies the exposition-only concept *cpp17-input-iterator*, `iterator_traits<I>` has the following publicly accessible members:

```
using iterator_category = see below;
using value_type        = typename indirectly_readable_traits<I>::value_type;
using difference_type   = typename incrementable_traits<I>::difference_type;
using pointer           = see below;
```

```
    using reference         = see below;
```

6. [*Example:* To implement a generic `reverse` function, a C++ program can do the following:

```
template<class BI>
void reverse(BI first, BI last) {
  typename iterator_traits<BI>::difference_type n = distance(first, last);
  --n;
  while(n > 0) {
    typename iterator_traits<BI>::value_type tmp = *first;
    *first++ = *--last;
    *last = tmp;
    n -= 2;
  }
}
```

— *end example*]

23.4.1 Standard iterator tags [std.iterator.tags]

3. [*Example:* If `evolve()` is well-defined for bidirectional iterators, but can be implemented more efficiently for random access iterators, then the implementation is as follows:

```
template<class BidirectionalIterator>
inline void
evolve(BidirectionalIterator first, BidirectionalIterator last) {
  evolve(first, last,
    typename iterator_traits<BidirectionalIterator>::iterator_category());
}

template<class BidirectionalIterator>
void evolve(BidirectionalIterator first, BidirectionalIterator last,
  bidirectional_iterator_tag) {
  // more generic, but less efficient algorithm
}

template<class RandomAccessIterator>
void evolve(RandomAccessIterator first, RandomAccessIterator last,
  random_access_iterator_tag) {
  // more efficient, but less generic algorithm
}
```

— *end example*]

23.4.2 Iterator operations [iterator.operations]

```
template<class InputIterator>
  constexpr typename iterator_traits<InputIterator>::difference_type
    distance(InputIterator first, InputIterator last);
```

4. *Preconditions:* `last` is reachable from `first`, or `InputIterator` meets the *Cpp17RandomAccessIterator* requirements and `first` is reachable from `last`.
5. *Effects:* If `InputIterator` meets the *Cpp17RandomAccessIterator* requirements, returns `(last - first)`; otherwise, returns the number of increments needed to get from `first` to `last`.

```
template<class InputIterator>
  constexpr InputIterator next(InputIterator x,
    typename iterator_traits<InputIterator>::difference_type n = 1);
```

6. *Effects:* Equivalent to: `advance(x, n); return x;`

```
template<class BidirectionalIterator>
  constexpr BidirectionalIterator prev(BidirectionalIterator x,
    typename iterator_traits<BidirectionalIterator>::difference_type n = 1);
```

7. *Effects:* Equivalent to: `advance(x, -n); return x;`

23.5.1.1 Class template `reverse_iterator` [reverse.iterator]

```
namespace std {
  template<class Iterator>
  class reverse_iterator {
  public:
    using iterator_type     = Iterator;
    using iterator_concept  = see below;
```

```
    using iterator_category = see below;
    using value_type        = iter_value_t<Iterator>;
    using difference_type   = iter_difference_t<Iterator>;
    using pointer           = typename iterator_traits<Iterator>::pointer;
    using reference         = iter_reference_t<Iterator>;

      // ...
  };
}
```

23.5.2.1 Class template `back_insert_iterator` [back.insert.iterator]

```
namespace std {
  template<class Container>
  class back_insert_iterator {
  protected:
    Container* container = nullptr;

  public:
    using iterator_category = output_iterator_tag;
    using value_type        = void;
    using difference_type   = ptrdiff_t;
    using pointer           = void;
    using reference         = void;
    using container_type    = Container;

    constexpr back_insert_iterator() noexcept = default;
    constexpr explicit back_insert_iterator(Container& x);
    constexpr back_insert_iterator& operator=(const typename Container::value_type& value);
    constexpr back_insert_iterator& operator=(typename Container::value_type&& value);

    constexpr back_insert_iterator& operator*();
    constexpr back_insert_iterator& operator++();
    constexpr back_insert_iterator  operator++(int);
  };

  template<class Container>
    back_insert_iterator<Container> back_inserter(Container& x);
}
```

23.5.2.1.1 Operations [back.insert.iter.ops]

```
back_insert_iterator& operator=(const typename Container::value_type& value);
```

2. *Effects:* As if by: `container->push_back(value);`
3. *Returns:* `*this`.

```
back_insert_iterator& operator=(typename Container::value_type&& value);
```

4. *Effects:* As if by: `container->push_back(std::move(value));`
5. *Returns:* `*this`.

23.5.2.2 Class template `front_insert_iterator` [front.insert.iterator]

```
namespace std {
  template<class Container>
  class front_insert_iterator {
  protected:
    Container* container = nullptr;

  public:
    using iterator_category = output_iterator_tag;
    using value_type        = void;
    using difference_type   = ptrdiff_t;
    using pointer           = void;
    using reference         = void;
    using container_type    = Container;

    constexpr front_insert_iterator() noexcept = default;
    constexpr explicit front_insert_iterator(Container& x);
    constexpr front_insert_iterator& operator=(const typename Container::value_type& value);
    constexpr front_insert_iterator& operator=(typename Container::value_type&& value);

    constexpr front_insert_iterator& operator*();
    constexpr front_insert_iterator& operator++();
    constexpr front_insert_iterator  operator++(int);
```

```
  };
}
```

23.5.2.2.1 Operations [front.insert.iter.ops]

```
front_insert_iterator& operator=(const typename Container::value_type& value);
```

2. *Effects:* As if by: `container->push_front(value);`
3. *Returns:* `*this`.

```
front_insert_iterator& operator=(typename Container::value_type&& value);
```

4. *Effects:* As if by: `container->push_front(std::move(value));`
5. *Returns:* `*this`.

23.5.2.3 Class template `insert_iterator` [insert.iterator]

```
namespace std {
  template<class Container>
  class insert_iterator {
  protected:
      Container* container = nullptr;
      ranges::iterator_t<Container> iter = ranges::iterator_t<Container>();

  public:
    using iterator_category = output_iterator_tag;
    using value_type        = void;
    using difference_type   = ptrdiff_t;
    using pointer           = void;
    using reference         = void;
    using container_type    = Container;

    insert_iterator() = default;
    constexpr insert_iterator(Container& x, ranges::iterator_t<Container> i);
    constexpr insert_iterator& operator=(const typename Container::value_type& value);
    constexpr insert_iterator& operator=(typename Container::value_type&& value);

    constexpr insert_iterator& operator*();
    constexpr insert_iterator& operator++();
    constexpr insert_iterator& operator++(int);
  };
}
```

23.5.2.3.1 Operations [insert.iter.ops]

```
insert_iterator(Container& x, ranges::iterator_t<Container> i);
```

1. *Effects:* Initializes `container` with `addressof(x)` and `iter` with `i`.

```
constexpr insert_iterator& operator=(const typename Container::value_type& value);
```

2. *Effects:* As if by:

```
iter = container->insert(iter, value);
++iter;
```

3. *Returns:* `*this`.

```
constexpr insert_iterator& operator=(typename Container::value_type&& value);
```

4. *Effects:* As if by:

```
iter = container->insert(iter, std::move(value));
++iter;
```

5. *Returns:* `*this`.

24.7.11.5 Class template `split_view::`*`inner-iterator`* [range.split.inner]

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct split_view<V, Pattern>::inner-iterator {
```

```
  private:
    using Base = conditional_t<Const, const V, V>;      // exposition only
    outer-iterator<Const> i_ = outer-iterator<Const>(); // exposition only
    bool incremented_ = false;                          // exposition only

  public:
    using iterator_concept  = typename outer-iterator<Const>::iterator_concept;
    using iterator_category = see below;
    using value_type        = range_value_t<Base>;
    using difference_type   = range_difference_t<Base>;

    // ...
  };

  // ...
}
```

24.7.15.3 Class template `elements_view::iterator` [range.elements.iterator]

```
namespace std::ranges {
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
  template<bool Const>
  class elements_view<V, N>::iterator {               // exposition only
    using Base = conditional_t<Const, const V, V>;    // exposition only

    iterator_t<Base> current_ = iterator_t<Base>();

  public:
    using iterator_category = typename iterator_traits<iterator_t<Base>>::iterator_category;
    using value_type = remove_cvref_t<tuple_element_t<N, range_value_t<Base>>>;
    using difference_type = range_difference_t<Base>;

    // ...
  };
}
```

25.4 Header `<algorithm>` synopsis [algorithm.syn]

```
#include <initializer_list>

namespace std {
  // ...

  // 25.6.9, count
  template<class InputIterator, class T>
    constexpr typename iterator_traits<InputIterator>::difference_type
      count(InputIterator first, InputIterator last, const T& value);
  template<class ExecutionPolicy, class ForwardIterator, class T>
    typename iterator_traits<ForwardIterator>::difference_type
      count(ExecutionPolicy&& exec, // see 25.3.5
            ForwardIterator first, ForwardIterator last, const T& value);
  template<class InputIterator, class Predicate>
    constexpr typename iterator_traits<InputIterator>::difference_type
      count_if(InputIterator first, InputIterator last, Predicate pred);
  template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    typename iterator_traits<ForwardIterator>::difference_type
      count_if(ExecutionPolicy&& exec, // see 25.3.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

  // ...

  // 25.7.14, shift
  template<class ForwardIterator>
    constexpr ForwardIterator
      shift_left(ForwardIterator first, ForwardIterator last,
                 typename iterator_traits<ForwardIterator>::difference_type n);
  template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator
      shift_left(ExecutionPolicy&& exec, // see 25.3.5
                 ForwardIterator first, ForwardIterator last,
                 typename iterator_traits<ForwardIterator>::difference_type n);
  template<class ForwardIterator>
    constexpr ForwardIterator
      shift_right(ForwardIterator first, ForwardIterator last,
                  typename iterator_traits<ForwardIterator>::difference_type n);
  template<class ExecutionPolicy, class ForwardIterator>
```

```
    ForwardIterator
      shift_right(ExecutionPolicy&& exec, // see 25.3.5
                  ForwardIterator first, ForwardIterator last,
                  typename iterator_traits<ForwardIterator>::difference_type n);

  // ...

}
```

25.6.9 Count [alg.count]

```
template<class InputIterator, class T>
  constexpr typename iterator_traits<InputIterator>::difference_type
    count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T>
  typename iterator_traits<ForwardIterator>::difference_type
    count(ExecutionPolicy&& exec,
          ForwardIterator first, ForwardIterator last, const T& value);

template<class InputIterator, class Predicate>
  constexpr typename iterator_traits<InputIterator>::difference_type
    count_if(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
  typename iterator_traits<ForwardIterator>::difference_type
    count_if(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last, Predicate pred);
```

1. *Effects:* Returns the number of iterators `i` in the range `[first, last)` for which the following corresponding conditions hold: `*i == value`, `pred(*i) != false`.
2. *Complexity:* Exactly `last - first` applications of the corresponding predicate.

25.9 Header `<numeric>` synopsis [numeric.ops.overview]

```
namespace std {
  // ...

  // 25.10.3, reduce
  template<class InputIterator>
    typename iterator_traits<InputIterator>::value_type
      reduce(InputIterator first, InputIterator last);
  template<class InputIterator, class T>
    T reduce(InputIterator first, InputIterator last, T init);
  template<class InputIterator, class T, class BinaryOperation>
    T reduce(InputIterator first, InputIterator last, T init, BinaryOperation binary_op);
  template<class ExecutionPolicy, class ForwardIterator>
    typename iterator_traits<ForwardIterator>::value_type
      reduce(ExecutionPolicy&& exec, // see 28.4.5 ForwardIterator first, ForwardIterator last);
  template<class ExecutionPolicy, class ForwardIterator, class T>
    T reduce(ExecutionPolicy&& exec, // see 28.4.5
             ForwardIterator first, ForwardIterator last, T init);
  template<class ExecutionPolicy, class ForwardIterator, class T, class BinaryOperation>
    T reduce(ExecutionPolicy&& exec, // see 28.4.5
             ForwardIterator first, ForwardIterator last, T init, BinaryOperation binary_op);

  // ...
}
```

25.10.3 Reduce [reduce]

```
template<class InputIterator>
  typename iterator_traits<InputIterator>::value_type
    reduce(InputIterator first, InputIterator last);
```

1. *Effects:* Equivalent to:

```
return reduce(first, last,
              typename iterator_traits<InputIterator>::value_type{});

template<class ExecutionPolicy, class ForwardIterator>
  typename iterator_traits<ForwardIterator>::value_type
    reduce(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator last);
```

2. *Effects:* Equivalent to:

```
return reduce(std::forward<ExecutionPolicy>(exec), first, last,
              typename iterator_traits<ForwardIterator>::value_type{});
```

25.11.2 `uninitialized_default_construct` [uninitialized.construct.default]

```
template <class NoThrowForwardIterator>
  void uninitialized_default_construct(NoThrowForwardIterator first, NoThrowForwardIterator last);
```

1. *Effects:* Equivalent to:

```
for (; first != last; ++first)
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type;
```

```
template <class NoThrowForwardIterator, class Size>
  NoThrowForwardIterator uninitialized_default_construct_n(NoThrowForwardIterator first, Size n);
```

3. *Effects:* Equivalent to:

```
for (; n > 0; (void)++first, --n)
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type;
return first;
```

25.11.3 `uninitialized_value_construct` [uninitialized.construct.value]

```
template <class NoThrowForwardIterator>
  void uninitialized_value_construct(NoThrowForwardIterator first, NoThrowForwardIterator last);
```

1. *Effects:* Equivalent to:

```
for (; first != last; ++first)
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type();
```

```
template <class NoThrowForwardIterator, class Size>
  NoThrowForwardIterator uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);
```

3. *Effects:* Equivalent to:

```
for (; n > 0; (void)++first, --n)
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type();
return first;
```

25.11.4 `uninitialized_copy` [uninitialized.copy]

```
template <class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                            NoThrowForwardIterator result);
```

1. *Preconditions:* `result + [0, (last - first))` does not overlap with `[first, last)`.
2. *Effects:* As if by:

```
for (; first != last; ++result, (void) ++first)
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type(*first);
```

3. *Returns:* `result`.

```
template <class InputIterator, class Size, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_copy_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

6. *Preconditions:* `result + [0, n)` does not overlap with `first + [0, n)`.
7. *Effects:* As if by:

```
for ( ; n > 0; ++result, (void) ++first, --n) {
  ::new (voidify(*first))
    typename iterator_traits<NoThrowForwardIterator>::value_type(*first);
}
```

8. *Returns:* `result`.

25.11.5 `uninitialized_move` [uninitialized.move]

```
template <class InputIterator, class ForwardIterator>
  NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                            NoThrowForwardIterator result);
```

1. *Preconditions:* `result + [0, (last - first))` does not overlap with `[first, last)`.
2. *Effects:* Equivalent to:

```
      for (; first != last; (void)++result, ++first)
        ::new (voidify(*first))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
      return result;

template <class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

6. *Preconditions:* `result + [0, n)` does not overlap with `first + [0, n)`.
7. *Effects:* Equivalent to:

```
      for (; n > 0; ++result, (void) ++first, --n)
        ::new (voidify(*first))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
      return {first,result};
```

23.11.6 `uninitialized_fill` [uninitialized.fill]

```
template <class NoThrowForwardIterator, class T>
  void uninitialized_fill(ForwardIterator first, NoThrowForwardIterator last, const T& x);
```

1. *Effects:* As if by:

```
      for (; first != last; ++first)
        ::new (voidify(*first))
          typename iterator_traits<NoThrowForwardIterator>::value_type(x);

template <class NoThrowForwardIterator, class Size, class T>
  ForwardIterator uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);
```

3. *Effects:* As if by:

```
      for (; n--; ++first)
        ::new (voidify(*first))
          typename iterator_traits<NoThrowForwardIterator>::value_type(x);
      return first;
```

26.6.4.2 Class template `discard_block_engine` [rand.adapt.disc]

3. The generation algorithm yields the value returned by the last invocation of `e()` while advancing `e`'s state as described above.

```
template<class Engine, size_t p, size_t r>
  class discard_block_engine {
  public:
    // types
    using result_type = typename Engine::result_type;

    // ...
};
```

26.6.4.4 Class template `shuffle_order_engine` [rand.adapt.shuf]

3. The generation algorithm yields the last value of `Y` produced while advancing `e`'s state as described above.

```
template<class Engine, size_t k>
  class shuffle_order_engine {
  public:
    // types
    using result_type = typename Engine::result_type;

    // ...
};
```

26.7.1 Header `<valarray>` synopsis [valarray.syn]

```
#include <initializer_list>
namespace std {
  template<class T> class valarray;        // An array of type T
  class slice;                             // a BLAS-like slice out of an array
  template<class T> class slice_array;
```

```
class gslice;                           // a generalized slice out of an array
template<class T> class gslice_array;
template<class T> class mask_array;     // a masked array
template<class T> class indirect_array; // an indirected array

template<class T> void swap(valarray<T>&, valarray<T>&) noexcept;

template<class T> valarray<T> operator* (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator* (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator* (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator/ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator/ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator/ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator% (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator% (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator+ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator+ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator- (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator- (const typename valarray<T>::value_type&,

template<class T> valarray<T> operator^ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator^ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator& (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator& (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator| (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator| (const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator<<(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator<<(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator<<(const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<T> operator>>(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator>>(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator>>(const typename valarray<T>::value_type&,
                                         const valarray<T>&);

template<class T> valarray<bool> operator&&(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&,
                                            const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator&&(const typename valarray<T>::value_type&,
                                            const valarray<T>&);

template<class T> valarray<bool> operator||(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator||(const valarray<T>&,
                                            const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator||(const typename valarray<T>::value_type&,
                                            const valarray<T>&);

template<class T> valarray<bool> operator==(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator==(const valarray<T>&,
```

```
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator==(const typename valarray<T>::value_type&,
                                          const valarray<T>&);
  template<class T> valarray<bool> operator!=(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator!=(const valarray<T>&,
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator!=(const typename valarray<T>::value_type&,
                                          const valarray<T>&);

  template<class T> valarray<bool> operator< (const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator< (const valarray<T>&,
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator< (const typename valarray<T>::value_type&,
                                          const valarray<T>&);
  template<class T> valarray<bool> operator> (const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator> (const valarray<T>&,
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator> (const typename valarray<T>::value_type&,
                                          const valarray<T>&);
  template<class T> valarray<bool> operator<=(const valarray<T>&, const valarray<T>&);

  template<class T> valarray<bool> operator<=(const valarray<T>&,
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator<=(const typename valarray<T>::value_type&,
                                          const valarray<T>&);
  template<class T> valarray<bool> operator>=(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator>=(const valarray<T>&,
                                          const typename valarray<T>::value_type&);
  template<class T> valarray<bool> operator>=(const typename valarray<T>::value_type&,
                                          const valarray<T>&);

  template<class T> valarray<T> abs  (const valarray<T>&);
  template<class T> valarray<T> acos (const valarray<T>&);
  template<class T> valarray<T> asin (const valarray<T>&);
  template<class T> valarray<T> atan (const valarray<T>&);

  template<class T> valarray<T> atan2(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> atan2(const valarray<T>&,
                                      const typename valarray<T>::value_type&);
  template<class T> valarray<T> atan2(const typename valarray<T>::value_type&,
                                      const valarray<T>&);

  template<class T> valarray<T> cos  (const valarray<T>&);
  template<class T> valarray<T> cosh (const valarray<T>&);
  template<class T> valarray<T> exp  (const valarray<T>&);
  template<class T> valarray<T> log  (const valarray<T>&);
  template<class T> valarray<T> log10(const valarray<T>&);

  template<class T> valarray<T> pow(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> pow(const valarray<T>&, const typename valarray<T>::value_type&);
  template<class T> valarray<T> pow(const typename valarray<T>::value_type&, const valarray<T>&);

  template<class T> valarray<T> sin  (const valarray<T>&);
  template<class T> valarray<T> sinh (const valarray<T>&);
  template<class T> valarray<T> sqrt (const valarray<T>&);
  template<class T> valarray<T> tan  (const valarray<T>&);
  template<class T> valarray<T> tanh (const valarray<T>&);

  template<class T> unspecified1 begin(valarray<T>& v);
  template<class T> unspecified2 begin(const valarray<T>& v);
  template<class T> unspecified1 end(valarray<T>& v);
  template<class T> unspecified2 end(const valarray<T>& v);
}
```

27.2 Header <chrono> synopsis [time.syn]

```
#include <compare> // see 17.11.1
namespace std {
  namespace chrono {
    // 27.5, class template duration
    template<class Rep, class Period = ratio<1>> class duration;

    // 27.6, class template time_point
    template<class Clock, class Duration = typename Clock::duration> class time_point;
  }

  // ...
}
```

27.5 Class template duration [time.duration]

```
namespace std::chrono {
  template<class Rep, class Period = ratio<1>>
    class duration {
    public:
      using rep    = Rep;
      using period = typename Period::type;

      // ...
    };
}
```

27.5.6 Comparisons [time.duration.comparisons]

```
    template<class Rep1, class Period1, class Rep2, class Period2>
        requires three_way_comparable<typename CT::rep>
      constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
                                 const duration<Rep2, Period2>& rhs);
```

7. *Returns:* `CT(lhs).count() <=> CT(rhs).count()`.

27.5.7 Conversions [time.duration.cast]

```
    template<class ToDuration, class Rep, class Period>
      constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

1. *Constraints:* `ToDuration` is a specialization of `duration`.
2. *Returns:* Let `CF` be `ratio_divide<Period, typename ToDuration::period>`, and `CR` be `common_type<typename ToDuration::rep, Rep, intmax_t>::type`.
    1. — If `CF::num == 1` and `CF::den == 1`, returns

        ```
        ToDuration(static_cast<typename ToDuration::rep>(d.count()))
        ```

    2. — otherwise, if `CF::num != 1` and `CF::den == 1`, returns

        ```
        ToDuration(static_cast<typename ToDuration::rep>(
          static_cast<CR>(d.count()) * static_cast<CR>(CF::num)))
        ```

    3. — otherwise, if `CF::num == 1` and `CF::den != 1`, returns

        ```
        ToDuration(static_cast<typename ToDuration::rep>(
          static_cast<CR>(d.count()) / static_cast<CR>(CF::den)))
        ```

    4. — otherwise, returns

        ```
        ToDuration(static_cast<typename ToDuration::rep>(
          static_cast<CR>(d.count()) * static_cast<CR>(CF::num) / static_cast<CR>(CF::den)))
        ```

```
    template<class ToDuration, class Rep, class Period>
      constexpr ToDuration round(const duration<Rep, Period>& d);
```

8. *Constraints:* `ToDuration` is a specialization of `duration`, and `treat_as_floating_point_v<typename ToDuration::rep>` is `false`.
9. *Returns:* The value of `ToDuration` that is closest to `d`. If there are two closest values, then return the value `t` for which `t % 2 == 0`.

27.6 Class template time_point [time.point]

```
    namespace std::chrono {
      template<class Clock, class Duration = typename Clock::duration>
        class time_point {
        public:
          using clock    = Clock;
          using duration = Duration;
          using rep      = typename duration::rep;
          using period   = typename duration::period;

          // ...
        };
    }
```

27.6.7 Conversions [time.point.cast]

```
template<class ToDuration, class Clock, class Duration>
  constexpr time_point<Clock, ToDuration> round(const time_point<Clock, Duration>& tp);
```

7. *Constraints:* `ToDuration` is a specialization of `duration`, and `treat_as_floating_point_v<typename ToDuration::rep>` is `false`.

27.7.1.3 Non-member functions [time.clock.system.nonmembers]

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);
```

1. *Constraints:* `treat_as_floating_point_v<typename Duration::rep>` is `false`, and `Duration{1} < days{1}` is `true`.

27.12 Formatting [time.format]

```
template<class Duration, class TimeZonePtr, class charT>
struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>
    : formatter<chrono::local-time-format-t<Duration>, charT> {
    template<class FormatContext>
      typename FormatContext::iterator
        format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx);
};
template<class FormatContext>
  typename FormatContext::iterator
    format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx);
```

16. *Effects:* Equivalent to:

```
sys_info info = tp.get_info();
return formatter<chrono::local-time-format-t<Duration>, charT>::
         format({tp.get_local_time(), &info.abbrev, &info.offset}, ctx);
```

28.3.1 `Class locale` [locale]

3. [*Example:* An iostream `operator<<` might be implemented as:[257]

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<< (basic_ostream<charT, traits>& s, Date d) {
  typename basic_ostream<charT, traits>::sentry cerberos(s);
  if (cerberos) {
    tm tmbuf; d.extract(tmbuf);
    bool failed =
      use_facet<time_put<charT, ostreambuf_iterator<charT, traits>>>(
        s.getloc()).put(s, s, s.fill(), &tmbuf, 'x').failed();
    if (failed)
      s.setstate(s.badbit);     // might throw
  }
  return s;
}
```

— *end example*]

28.4.1.2 Class template ctype_byname [locale.ctype.byname]

```
namespace std {
  emplate<class charT>
    class ctype_byname : public ctype<charT> {
    public:
      using mask = typename ctype<charT>::mask;
      explicit ctype_byname(const char*, size_t refs = 0);
      explicit ctype_byname(const string&, size_t refs = 0);

    protected:
      ~ctype_byname();

    };
}
```

29.5.5.1 Overview [ios.overview]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_ios : public ios_base {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.5.5.4, flags functions ...
    };
}
```

29.6.3 Class template `basic_streambuf` [streambuf]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_streambuf {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      virtual ~basic_streambuf();

      // 29.6.3.2.1, locales ...
    };
}
```

29.7.4.1 Class template `basic_istream` [istream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_istream : virtual public basic_ios<charT, traits> {
    public:
      // types (inherited from basic_ios (29.5.5))
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.7.4.1.1, constructor/destructor ...
    };
}
```

29.7.4.6 Class template `basic_iostream` [iostream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_iostream
      : public basic_istream<charT, traits>,
        public basic_ostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.7.4.6.1, constructor ...
    };
}
```

29.7.5.1 Class template `basic_ostream` [ostream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_ostream : virtual public basic_ios<charT, traits> {
    public:
      // types (inherited from basic_ios (29.5.5))
      using char_type      = charT;
      using int_type       = typename traits::int_type;
```

```
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.7.5.1.1, constructor/destructor ...
    };
}
```

29.8.2 Class template `basic_stringbuf` [stringbuf]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_stringbuf : public basic_streambuf<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.8.2.1, constructors/destructor
    };
}
```

29.8.3 Class template `basic_istringstream` [istringstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_istringstream : public basic_istream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.8.3.1, constructors ...
    };
}
```

29.8.4 Class template `basic_ostringstream` [ostringstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_ostringstream : public basic_ostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.8.4.1, constructors ...
    };
}
```

29.8.5 Class template `basic_stringstream` [stringstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_stringstream : public basic_iostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.8.5.1, constructors ...
    };
}
```

29.9.2 Class template `basic_filebuf` [filebuf]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_filebuf : public basic_streambuf<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.9.2.1, constructors/destructor
    };
}
```

5. In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as a_codecvt in following subclauses, obtained as if by

```
const codecvt<charT, char, typename traits::state_type>& a_codecvt =
  use_facet<codecvt<charT, char, typename traits::state_type>>(getloc());
```

29.9.3 Class template `basic_ifstream` [ifstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_ifstream : public basic_istream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;

      // 29.9.3.1, constructors ...
    };
}
```

29.9.4 Class template `basic_ofstream` [ofstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_ofstream : public basic_ostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.9.4.1, constructors ...
    };
}
```

29.9.5 Class template `basic_fstream` [fstream]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
    class basic_fstream : public basic_iostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      // 29.9.5.1, constructors ...
    };
}
```

29.10.2.1 Overview [syncstream.syncbuf.overview]

```
namespace std {
  template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
    class basic_syncbuf : public basic_streambuf<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
```

```
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;
      using allocator_type = Allocator;

      using streambuf_type = basic_streambuf<charT, traits>;

      // 29.10.2.2, construction and destruction...
    };
}
```

29.10.3.1 Overview [syncstream.osyncstream.overview]

```
namespace std {
  template<class charT, class traits = char_traits<charT>, class Allocator = allocator>charT>>
    class basic_osyncstream : public basic_ostream<charT, traits> {
    public:
      using char_type      = charT;
      using int_type       = typename traits::int_type;
      using pos_type       = typename traits::pos_type;
      using off_type       = typename traits::off_type;
      using traits_type    = traits;;

      using allocator_type = Allocator;
      using streambuf_type = basic_streambuf<charT, traits>;
      using syncbuf_type   = basic_syncbuf<charT, traits, Allocator>;

      // 29.10.3.2, construction and destruction ...
    };
}
```

30.4 Header <regex> synopsis [re.syn]

```
#include <compare>          // see 17.11.1
#include <initializer_list> // see 17.10.1

namespace std {
  // 30.5, regex constants
  namespace regex_constants {
    using syntax_option_type = T1;
    using match_flag_type = T2;
    using error_type = T3;
  }

  // 30.6, class regex_error
  class regex_error;

  // 30.7, class template regex_traits
  template<class charT> struct regex_traits;

  // 30.8, class template basic_regex
  template<class charT, class traits = regex_traits<charT>>
    class basic_regex;

  using regex  = basic_regex<char>;
  using wregex = basic_regex<wchar_t>;

  // 30.8.5, basic_regex swap
  template<class charT, class traits>
    void swap(basic_regex<charT, traits>& e1, basic_regex<charT, traits>& e2);

  // 30.9, class template sub_match
  template<class BidirectionalIterator>
    class sub_match;

  using csub_match  = sub_match<const char*>;
  using wcsub_match = sub_match<const wchar_t*>;
  using ssub_match  = sub_match<string::const_iterator>;
  using wssub_match = sub_match<wstring::const_iterator>;

  // 30.9.2, sub_match non-member operators
  template<class BiIter>
    bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
  template<class BiIter>
    auto operator<=>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
  template<class BiIter, class ST, class SA>
    bool operator==(
      const sub_match<BiIter>& lhs,
```

```
    const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
  auto operator<=>(
    const sub_match<BiIter>& lhs,
    const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
  auto operator<=>(const sub_match<BiIter>& lhs,
                   const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type& rhs);
template<class BiIter>
  auto operator<=>(const sub_match<BiIter>& lhs,
                   const typename iterator_traits<BiIter>::value_type& rhs);
template<class charT, class ST, class BiIter>
  basic_ostream<charT, ST>&
    operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);

// 30.10, class template match_results
template<class BidirectionalIterator,
         class Allocator = allocator<sub_match<BidirectionalIterator>>>
  class match_results;

using cmatch  = match_results<const char*>;
using wcmatch = match_results<const wchar_t*>;
using smatch  = match_results<string::const_iterator>;
using wsmatch = match_results<wstring::const_iterator>;

// match_results comparisons
template<class BidirectionalIterator, class Allocator>
  bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
                  const match_results<BidirectionalIterator, Allocator>& m2);

// 30.10.7, match_results swap
template<class BidirectionalIterator, class Allocator>
  void swap(match_results<BidirectionalIterator, Allocator>& m1,
            match_results<BidirectionalIterator, Allocator>& m2);

// 30.11.2, function template regex_match
template<class BidirectionalIterator, class Allocator, class charT, class traits>
  bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                   match_results<BidirectionalIterator, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
  bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class Allocator, class traits>
  bool regex_match(const charT* str, match_results<const charT*, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                   match_results<typename basic_string<charT, ST, SA>::const_iterator,
                                 Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>&&,
                   match_results<typename basic_string<charT, ST, SA>::const_iterator,
                                 Allocator>&,
                   const basic_regex<charT, traits>&,
                   regex_constants::match_flag_type = regex_constants::match_default) = delete;
template<class charT, class traits>
  bool regex_match(const charT* str,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags = regex_constants::match_default);

// 30.11.3, function template regex_search
template<class BidirectionalIterator, class Allocator, class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                    match_results<BidirectionalIterator, Allocator>& m,
```

```
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class Allocator, class traits>
  bool regex_search(const charT* str,
                    match_results<const charT*, Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class traits>
  bool regex_search(const charT* str,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);

template<class ST, class SA, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    match_results<typename basic_string<charT, ST, SA>::const_iterator,
                                  Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>&&,
                    match_results<typename basic_string<charT, ST, SA>::const_iterator,
                                  Allocator>&,
                    const basic_regex<charT, traits>&,
                    regex_constants::match_flag_type
                      = regex_constants::match_default) = delete;

// 30.11.4, function template regex_replace
template<class OutputIterator, class BidirectionalIterator,
         class traits, class charT, class ST, class SA>
  OutputIterator
    regex_replace(OutputIterator out,
                  BidirectionalIterator first, BidirectionalIterator last,
                  const basic_regex<charT, traits>& e,
                  const basic_string<charT, ST, SA>& fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);
template<class OutputIterator, class BidirectionalIterator, class traits, class charT>
  OutputIterator
    regex_replace(OutputIterator out,
                  BidirectionalIterator first, BidirectionalIterator last,
                  const basic_regex<charT, traits>& e,
                  const charT* fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA, class FST, class FSA>
  basic_string<charT, ST, SA>
    regex_replace(const basic_string<charT, ST, SA>& s,
                  const basic_regex<charT, traits>& e,
                  const basic_string<charT, FST, FSA>& fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA>
  basic_string<charT, ST, SA>
    regex_replace(const basic_string<charT, ST, SA>& s,
                  const basic_regex<charT, traits>& e,
                  const charT* fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA>
  basic_string<charT>
    regex_replace(const charT* s,
                  const basic_regex<charT, traits>& e,
                  const basic_string<charT, ST, SA>& fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT>
  basic_string<charT>
    regex_replace(const charT* s,
                  const basic_regex<charT, traits>& e,
                  const charT* fmt,
                  regex_constants::match_flag_type flags = regex_constants::match_default);

// 30.12.1, class template regex_iterator
template<class BidirectionalIterator,
         class charT = typename iterator_traits<BidirectionalIterator>::value_type,
         class traits = regex_traits<charT>>
  class regex_iterator;
```

```
  using cregex_iterator  = regex_iterator<const char*>;
  using wcregex_iterator = regex_iterator<const wchar_t*>;
  using sregex_iterator  = regex_iterator<string::const_iterator>;
  using wsregex_iterator = regex_iterator<wstring::const_iterator>;

  // 30.12.2, class template regex_token_iterator
  template<class BidirectionalIterator,
           class charT = typename iterator_traits<BidirectionalIterator>::value_type,
           class traits = regex_traits<charT>>
    class regex_token_iterator;

    using cregex_token_iterator  = regex_token_iterator<const char*>;
    using wcregex_token_iterator = regex_token_iterator<const wchar_t*>;
    using sregex_token_iterator  = regex_token_iterator<string::const_iterator>;
    using wsregex_token_iterator = regex_token_iterator<wstring::const_iterator>;

    namespace pmr {
      template<class BidirectionalIterator>
        using match_results =
          std::match_results<BidirectionalIterator,
                             polymorphic_allocator<sub_match<BidirectionalIterator>>>;

      using cmatch  = match_results<const char*>;
      using wcmatch = match_results<const wchar_t*>;
      using smatch  = match_results<string::const_iterator>;
      using wsmatch = match_results<wstring::const_iterator>;
    }
  }
```

30.8 Class template `basic_regex` [re.regex]

```
  namespace std {
    template<class charT, class traits = regex_traits<charT>>
      class basic_regex {
      public:
        // types
        using value_type  =          charT;
        using traits_type =          traits;
        using string_type = typename traits::string_type;
        using flag_type   =          regex_constants::syntax_option_type;
        using locale_type = typename traits::locale_type;

        // 30.5.1, constants ...

      };

    template<class ForwardIterator>
      basic_regex(ForwardIterator, ForwardIterator,
                  regex_constants::syntax_option_type = regex_constants::ECMAScript)
                  -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
  }
```

30.9 Class template sub_match [re.submatch]

1. Class template `sub_match` denotes the sequence of characters matched by a particular marked sub-expression

```
  namespace std {
    template<class BidirectionalIterator>
      class sub_match : public pair<BidirectionalIterator, BidirectionalIterator> {
      public:
        using value_type      =
                typename iterator_traits<BidirectionalIterator>::value_type;
        using difference_type =
                typename iterator_traits<BidirectionalIterator>::difference_type;
        using iterator        = BidirectionalIterator;
        using string_type     = basic_string<value_type>;

        bool matched;

        constexpr sub_match();

        difference_type length() const;
        operator string_type() const;
        string_type str() const;

        int compare(const sub_match& s) const;
        int compare(const string_type& s) const;
```

```
      int compare(const value_type* s) const;
};
```

30.10 Class template `match_results` [re.results]

```
namespace std {
  template<class BidirectionalIterator,
           class Allocator = allocator<sub_match<BidirectionalIterator>>>
    class match_results {
    public:
      using value_type      = sub_match<BidirectionalIterator>;
      using const_reference = const value_type&;
      using reference       = value_type&;
      using const_iterator  = implementation-defined;
      using iterator        = const_iterator;
      using difference_type =
        typename iterator_traits<BidirectionalIterator>::difference_type;
      using size_type       = typename allocator_traits<Allocator>::size_type;
      using allocator_type  = Allocator;
      using char_type       =
        typename iterator_traits<BidirectionalIterator>::value_type;
      using string_type     = basic_string<char_type>;

      // ...
  };
}
```

30.12.1 Class template `regex_iterator` [re.regiter]

1. The class template `regex_iterator` is an iterator adaptor. It represents ...

```
namespace std {
  template<class BidirectionalIterator,
           class charT = typename iterator_traits<BidirectionalIterator>::value_type,
           class traits = regex_traits<charT>>
    class regex_iterator {
      // ...
    };
}
```

30.12.2 Class template `regex_token_iterator` [re.tokiter]

6. It is impossible to store things into `regex_token_iterators`. Two end-of-sequence iterators ...

```
namespace std {
  template<class BidirectionalIterator,
           class charT = typename iterator_traits<BidirectionalIterator>::value_type,
           class traits = regex_traits<charT>>
    class regex_token_iterators {
      // ...
    };
}
```

31.2 Header `<atomic>` synopsis [atomics.syn]

```
namespace std {
  // ...

  // 31.9, non-member functions
  template<class T>
    bool atomic_is_lock_free(const volatile atomic<T>*) noexcept;
  template<class T>
    bool atomic_is_lock_free(const atomic<T>*) noexcept;

  template<class T>
    void atomic_store(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    void atomic_store(atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    void atomic_store_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
  template<class T>
    void atomic_store_explicit(atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
  template<class T>
    T atomic_load(const volatile atomic<T>*) noexcept;
```

```
template<class T>
  T atomic_load(const atomic<T>*) noexcept;
template<class T>
  T atomic_load_explicit(const volatile atomic<T>*, memory_order) noexcept;
template<class T>
  T atomic_load_explicit(const atomic<T>*, memory_order) noexcept;
template<class T>
  T atomic_exchange(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  T atomic_exchange(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  T atomic_exchange_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                             memory_order) noexcept;
template<class T>
  T atomic_exchange_explicit(atomic<T>*, typename atomic<T>::value_type,
                             memory_order) noexcept;
template<class T>
  bool atomic_compare_exchange_weak(volatile atomic<T>*,
                                    typename atomic<T>::value_type*,
                                    typename atomic<T>::value_type) noexcept;
template<class T>
  bool atomic_compare_exchange_weak(atomic<T>*,
                                    typename atomic<T>::value_type*,
                                    typename atomic<T>::value_type) noexcept;
template<class T>
  bool atomic_compare_exchange_strong(volatile atomic<T>*,
                                      typename atomic<T>::value_type*,
                                      typename atomic<T>::value_type) noexcept;
template<class T>
  bool atomic_compare_exchange_strong(atomic<T>*,
                                      typename atomic<T>::value_type*,
                                      typename atomic<T>::value_type) noexcept;
template<class T>
  bool atomic_compare_exchange_weak_explicit(volatile atomic<T>*,
                                             typename atomic<T>::value_type*,
                                             typename atomic<T>::value_type,
                                             memory_order, memory_order) noexcept;
template<class T>
  bool atomic_compare_exchange_weak_explicit(atomic<T>*,
                                             typename atomic<T>::value_type*,
                                             typename atomic<T>::value_type,
                                             memory_order, memory_order) noexcept;
template<class T>
  bool atomic_compare_exchange_strong_explicit(volatile atomic<T>*,
                                               typename atomic<T>::value_type*,
                                               typename atomic<T>::value_type,
                                               memory_order, memory_order) noexcept;
template<class T>
  bool atomic_compare_exchange_strong_explicit(atomic<T>*,
                                               typename atomic<T>::value_type*,
                                               typename atomic<T>::value_type,
                                               memory_order, memory_order) noexcept;

template<class T>
  T atomic_fetch_add(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
  T atomic_fetch_add(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
  T atomic_fetch_add_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
                              memory_order) noexcept;
template<class T>
  T atomic_fetch_add_explicit(atomic<T>*, typename atomic<T>::difference_type,
                              memory_order) noexcept;
template<class T>
  T atomic_fetch_sub(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
  T atomic_fetch_sub(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
  T atomic_fetch_sub_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
                              memory_order) noexcept;
template<class T>
  T atomic_fetch_sub_explicit(atomic<T>*, typename atomic<T>::difference_type,
                              memory_order) noexcept;
template<class T>
  T atomic_fetch_and(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  T atomic_fetch_and(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
  T atomic_fetch_and_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                              memory_order) noexcept;
```

```
  template<class T>
    T atomic_fetch_and_explicit(atomic<T>*, typename atomic<T>::value_type,
                                memory_order) noexcept;
  template<class T>
    T atomic_fetch_or(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    T atomic_fetch_or(atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    T atomic_fetch_or_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
  template<class T>
    T atomic_fetch_or_explicit(atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;
  template<class T>
    T atomic_fetch_xor(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    T atomic_fetch_xor(atomic<T>*, typename atomic<T>::value_type) noexcept;
  template<class T>
    T atomic_fetch_xor_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                                memory_order) noexcept;
  template<class T>
    T atomic_fetch_xor_explicit(atomic<T>*, typename atomic<T>::value_type,
                                memory_order) noexcept;
  template<class T>
    void atomic_wait(const volatile atomic<T>*, typename atomic<T>::value_type);
  template<class T>
    void atomic_wait(const atomic<T>*, typename atomic<T>::value_type);
  template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*, typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_wait_explicit(const atomic<T>*, typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_notify_one(volatile atomic<T>*);
  template<class T>
    void atomic_notify_one(atomic<T>*);
  template<class T>
    void atomic_notify_all(volatile atomic<T>*);
  template<class T>
    void atomic_notify_all(atomic<T>*);

  // 31.3, type aliases ...

}
```

6 Acknowledgements

Thanks to Richard Smith and Daveed Vandevoorde for review of early versions of this paper that caught several technical errors. Thanks to Jens Maurer for suggesting to list each pattern in section 3, that really improved the clarity of the paper. Thanks to Dietmar Kühl and Daryl Haresign for a final review of the paper that again, greatly improved its clarity.

7 References

- N4861 C++ Standard Working Draft (at inception of C++23), Richard Smith
- Core #1008 Querying the alignment of an object
- P0634R3 Down with `typename`!, Nina Ranns, Daveed Vandevoorde
- P0945R0 Generalizing alias declarations, Richard Smith